Bölüm 3: Assembly

Mikroişlemciler

Assembly dili | Basit bilgisayar modeli | Sistem yolu | CPU'nun içi | Genel amaçlı yazmaçlar | Kesim yazmaçları | Fiziksel adres | IP ve bayraklar | İlk program

Sercan KÜLCÜ · Giresun Üniversitesi · Elektrik-Elektronik Mühendisliği

GİRİŞ

Öğrenme Çıktıları

Assembly'nin yeri
Assembly dilinin makine dili ve yüksek seviyeli dillerle ilişkisini açıklamak.

Bilgisayar modeli
CPU, bellek, G/Ç aygıtları ve sistem yolunun görevlerini tanımlamak.

8086 yazmaçları
Genel amaçlı, kesim ve özel amaçlı yazmaçları ve 8/16 bit ilişkisini kullanmak.

Adres hesabı
Kesim:bağıl konum çiftinden 20 bitlik fiziksel adresi hesaplamak.

IP ve bayraklar
IP'nin nasıl ilerlediğini ve bir işlemin bayrakları nasıl etkilediğini izlemek.

İlk program
emu8086 ile basit bir .COM programını yazıp adım adım çalıştırmak.

Assembly Dili

- **Düşük seviyeli** bir programlama dilidir: her komut, işlemcinin tek bir makine komutuna karşılık gelir.
- Makine kodundaki sayılar yerine akılda kalıcı **komut kısaltmaları** (mnemonic) kullanılır: `MOV`, `ADD`, `JMP`...
- Bilgisayarın yapısı (yazmaçlar, bellek, sistem yolu) hakkında **bilgi sahibi olmayı** gerektirir.
- Bilgisayarın temel yapısını ve bir programın gerçekte nasıl çalıştığını **anlamak** açısından önemlidir.
- Her işlemci ailesinin kendi assembly dili vardır; bu derste **Intel 8086** ve **emu8086** kullanılır.

Tek satır, üç görünüm

Assembly: `MOV AX, 5`

Onaltılık makine kodu: `B8 05 00`

CPU'nun gördüğü:

```
10111000 00000101 00000000
```

16 bitlik 0005h, bellekte düşük bayt önce (05 00) saklanır.

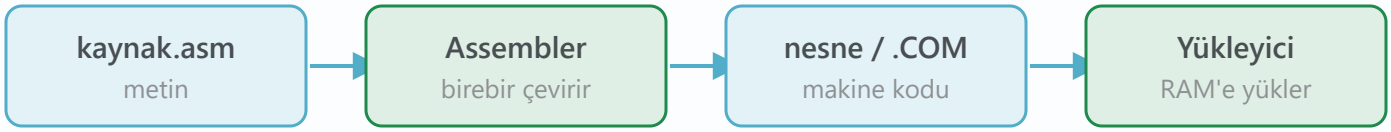
Dil Düzeyleri

Düzey	Örnek	Kim anlar?
Yüksek seviyeli dil (C)	`x = 5; x++; return;`	İnsan; derleyici makine koduna çevirir
Assembly	`MOV AX, 5` · `INC AX` · `RET`	İnsan; *assembler* birebir çevirir
Makine dili (onaltılık)	`B8 05 00` · `40` · `C3`	CPU (ikili olarak)
Makine dili (ikili)	`10111000 00000101 00000000` · `01000000` · `11000011`	CPU

Bir C satırı çoğu zaman **birçok** makine komutuna dönüşür; bir assembly satırı ise (direktifler hariç) **tam olarak bir** makine komutudur. Bu yüzden assembly, işlemcinin "ne yaptığını" doğrudan gösterir.

ASSEMBLY DİLİ

Assembler ve emu8086



EXE için araya bağlayıcı (linker) girer: .obj → .exe

Assembler

Komut kısaltmalarını ve etiketleri makine koduna, değişken adlarını adreslere çevirir. Örnek: MASM, TASM, NASM.

emu8086

Editör + assembler + 8086 öykünücüsü (emulator). Yazmaçları, belleği ve bayrakları *adım adım* gösterir.

.COM / .EXE

.COM tek kesimli, en fazla 64 KB'lık basit biçimdir (ORG 100h). .EXE çok kesimli olabilir.

Bu derste örnekler emu8086 söz dizimiyle yazılır; çoğu MASM/TASM ile de uyumludur.

Neden Assembly?

Assembly dili ile oluşturulmuş bir program:

Artıları

+ Daha **az yer** kaplar (gereksiz kod yok).
+ Yalnızca yapması gereken işlemleri yaptığından **daha hızlıdır**.
+ Yüksek seviyeli dillerin yapamadığı işlemleri yapabilir: port G/Ç, kesme işleme, bayraklara doğrudan erişim.
+ Önyükleyici, aygıt sürücüsü, gömülü sistem gibi alanlarda vazgeçilmezdir.

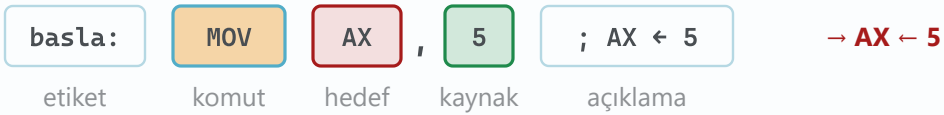
Eksileri

– **Taşınabilir değildir**: 8086 kodu ARM'da çalışmaz.
– Yazması, okuması ve hata ayıklaması zahmetlidir.
– Büyük projelerde verim düşüktür; modern derleyiciler çoğu zaman yeterince hızlı kod üretir.

Uygulamada genellikle **karma** yaklaşım kullanılır: program yüksek seviyeli dilde, zamanlaması kritik küçük parçalar assembly ile yazılır.

Komut Satırının Yapısı

```
[etiket:]  komut  [hedef], [kaynak]  [; açıklama]
```



Satır	Etiket	Komut	İşlenenler	Açıklama
`basla: MOV AX, 5 ; AX ← 5`	`basla`	`MOV`	`AX`, `5`	`; AX ← 5`
`INC AX`	—	`INC`	`AX`	—
`RET`	—	`RET`	yok	—

- İki işlenenli komutlarda sıra **hedef, kaynak**tır: `MOV AX, BX` → AX ← BX.
- Assembler büyük-küçük harfe duyarsızdır: `mov ax, bx` ile `MOV AX, BX` aynıdır.
- `;` işaretinden satır sonuna kadar her şey açıklamadır ve makine koduna çevrilmez.

Komut mu, Direktif mi?

Satır	Türü	Bellekte ne üretir?
`MOV AX, 5`	komut	makine kodu: B8 05 00
`ORG 100h`	direktif	hiçbir şey; konum sayacını 0100h yapar
`sayi DB 0`	direktif (veri tanımı)	1 bayt veri: 00
`k EQU 5`	direktif	hiçbir şey; k adı 5 değerini alır
`END`	direktif	hiçbir şey; kaynak burada biter
`PUTC 'A'`	makro çağrısı	makro gövdesindeki komutların kodu

Komut

İşlemcinin çalışma anında yürüttüğü işlem. Her biri bir makine komutuna çevrilir.

Direktif

Assembler'a verilen bilgi: nereye yerleşecek, ne kadar yer ayrılacak, adlar neyi gösterecek.

Makro

Derleme anında yerine birkaç satır yazılan, adı olan metin kalıbı.

Etiketler ve Adlar

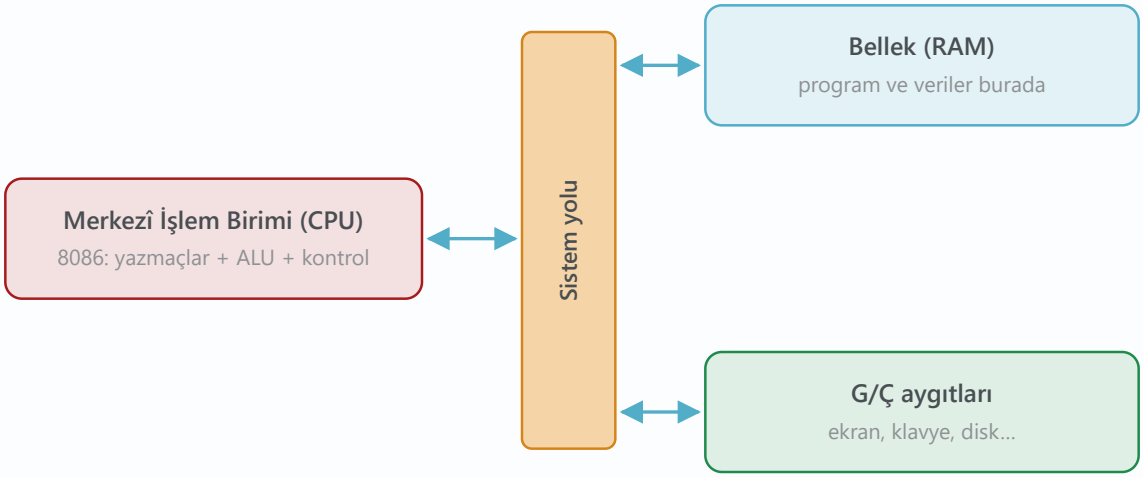
```
basla:  MOV CX, 3    ; kod etiketi
don:    INC AX
        LOOP don     ; don = adres
        RET
sayi    DB 7         ; veri adı
```

- Kod etiketi iki nokta üst üste ile biter ve bir komutun adresini (CS içindeki bağıl konumunu) temsil eder; kendisi makine kodu üretmez.
- Değişken adı iki noktasız yazılır; adı, türü (DB, DW…) ve adresi birlikte saklanır.
- Ad harfle ya da alt çizgiyle başlar, rakamla başlayamaz; devamında harf, rakam ve alt çizgi olabilir.
- Komut, yazmaç ve direktif adları (MOV, AX, DB, END…) ad olarak kullanılamaz.
- Büyük-küçük harf ayrılmaz: `don` ile `DON` aynı etikettir; ikisini birden tanımlamak “iki kez tanımlı” hatası verir.

Ad	`_sayac`	`toplam2`	`2toplam`	`cx`	`son-deger`
Geçerli mi?	evet	evet	rakamla başlıyor	yazmaç adı	– bir işleçtir

BİLGİSAYAR MODELİ

Basit Bilgisayar Modeli



Sistem yolu
Bilgisayarın bileşenlerini birbirine bağlar.

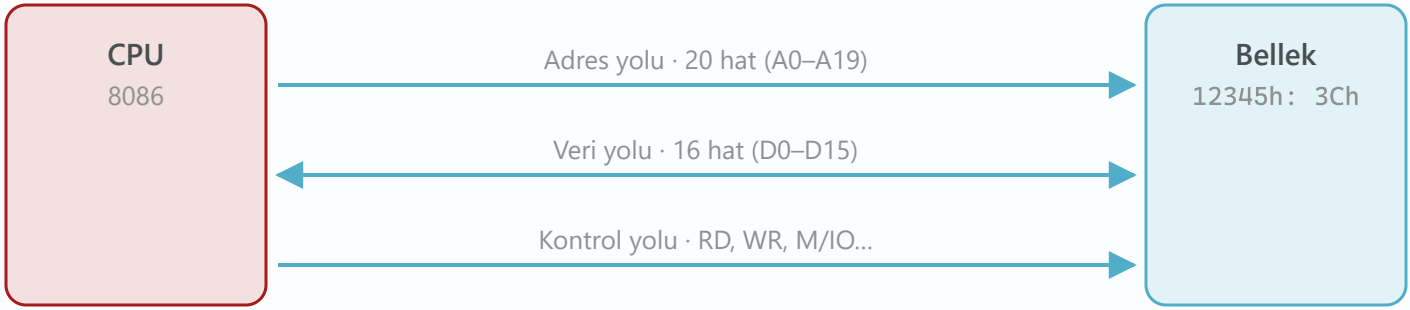
CPU
Bilgisayarın kalbi; hesaplamaların çoğu CPU içinde yapılır.

RAM
Programların yürütülmek üzere yüklendiği alan: komutlar ve veriler.

G/Ç aygıtları
Kullanıcıyla ve dış dünyayla iletişim.

BİLGİSAYAR MODELİ

Sistem Yolu



Adres yolu

CPU'dan belleğe/G/Ç'ye tek yönlü. 8086'da 20 hat → 2^{20} = **1 MB** adreslenebilir bellek.

Veri yolu

Çift yönlü. 8086'da 16 hat → tek seferde 1 bayt ya da 1 sözcük (word) aktarılır.

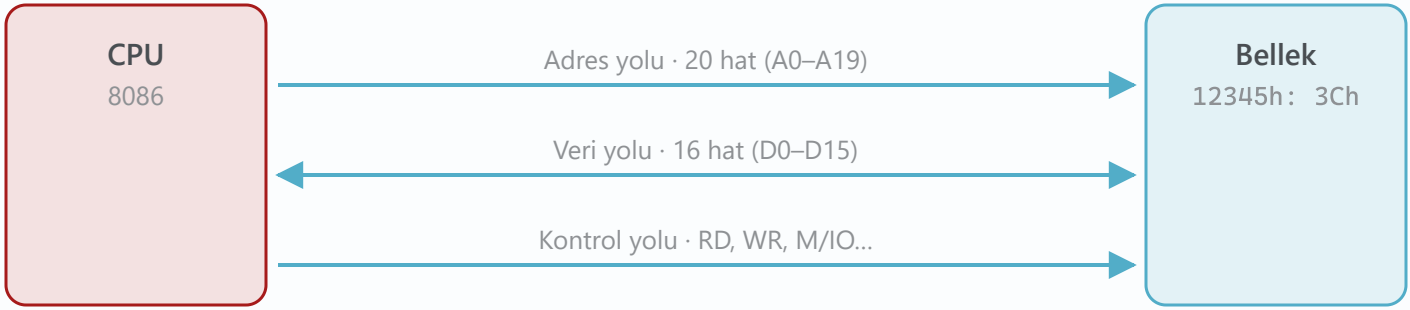
Kontrol yolu

İşlemin türünü bildirir: okuma (RD), yazma (WR), bellek mi G/Ç mı (M/IO), saat, kesme…

8086'da adres ve veri hatları aynı uçlardan zaman paylaşımlı geçer (AD0–AD15).

BİLGİSAYAR MODELİ

Bellekten Okuma Adım Adım



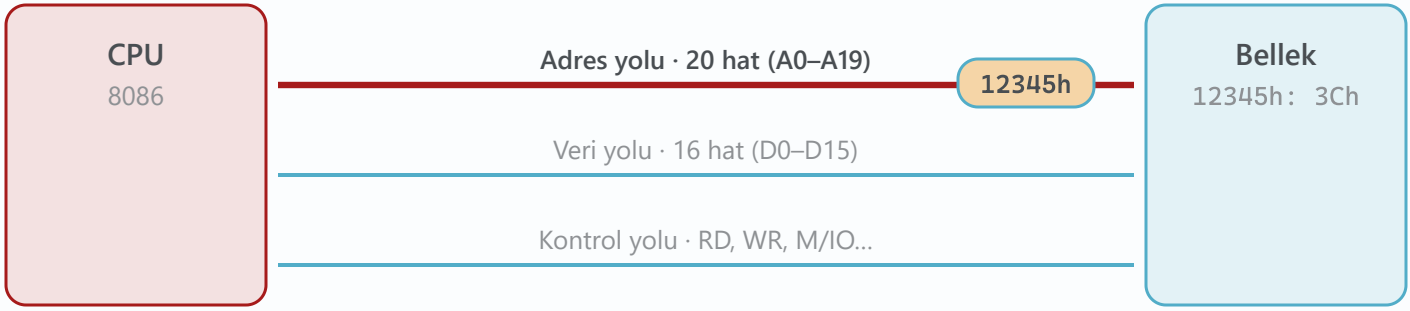
Komut: `MOV AL, [12345h fiziksel adresi]`. CPU'nun bir bayta ihtiyacı var; veri bellekte.

Adım 1 / 5

Yazma döngüsünde 3. adımda veriyi CPU koyar ve WR sinyali etkin olur.

BİLGİSAYAR MODELİ

Bellekten Okuma Adım Adım



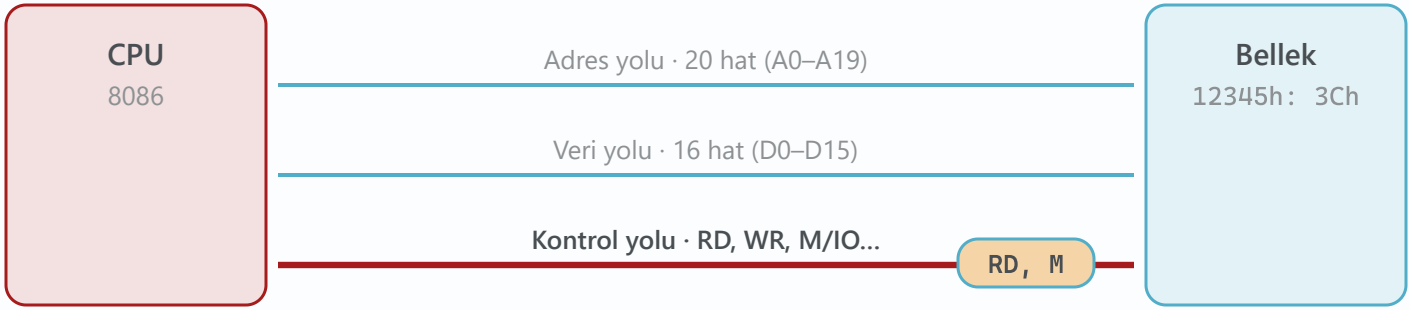
1. CPU, okunacak adresi **adres yoluna** koyar: 12345h.

Adım 2 / 5

Yazma döngüsünde 3. adımda veriyi CPU koyar ve WR sinyali etkin olur.

BİLGİSAYAR MODELİ

Bellekten Okuma Adım Adım



2. Kontrol yolu: “bellekten oku” (RD etkin, M/IO = bellek).

Adım 3 / 5

Yazma döngüsünde 3. adımda veriyi CPU koyar ve WR sinyali etkin olur.

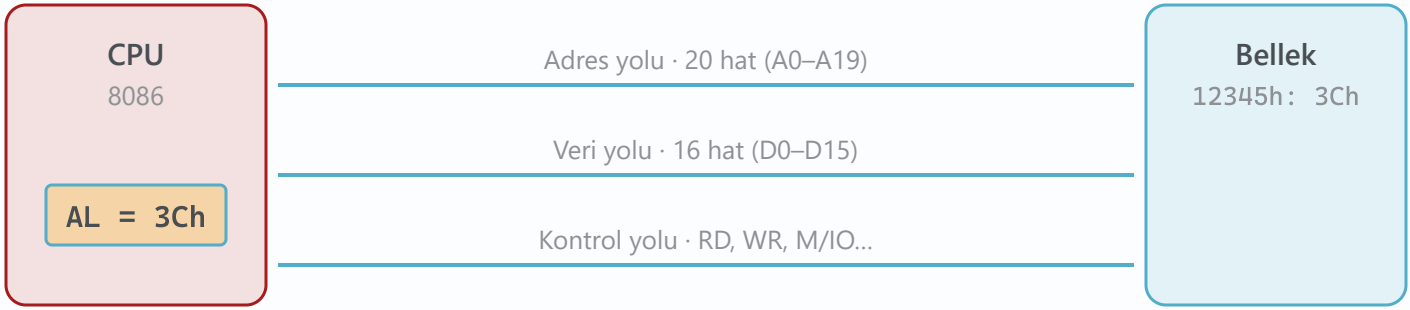
Bellekten Okuma Adım Adım



3. Bellek, 12345h'deki baytı (3Ch) **veri yoluna** koyar.

Yazma döngüsünde 3. adımda veriyi CPU koyar ve WR sinyali etkin olur.

Bellekten Okuma Adım Adım



4. CPU veriyi alır ve hedef yazmaca yazar: **AL = 3Ch**. 8086'da bu bir **yol döngüsü**dür (4 saat periyodu: T1–T4).

Adım 5 / 5

Yazma döngüsünde 3. adımda veriyi CPU koyar ve WR sinyali etkin olur.

BİLGİSAYAR MODELİ

Getir – Çöz – Yürüt



Getir: CPU, CS:IP adresindeki komut baytlarını bellekten okur ve **IP'yi komut uzunluğu kadar artırır**.

Adım 1 / 3

8086'da **BIU** (yol arayüz birimi) komutları önden getirip 6 baytlık **komut kuyruğuna** koyar; **EU** (yürütme birimi) bu sırada önceki komutu yürütür. Bu basit bir boru hattıdır (pipelining).

BİLGİSAYAR MODELİ

Getir – Çöz – Yürüt



Çöz: İşlem kodu (opcode) çözülür: hangi işlem, hangi yazmaç/bellek işlenenleri?

Adım 2 / 3

8086'da **BIU** (yol arayüz birimi) komutları önden getirip 6 baytlık **komut kuyruğuna** koyar; **EU** (yürütme birimi) bu sırada önceki komutu yürütür. Bu basit bir boru hattıdır (pipelining).

BİLGİSAYAR MODELİ

Getir – Çöz – Yürüt

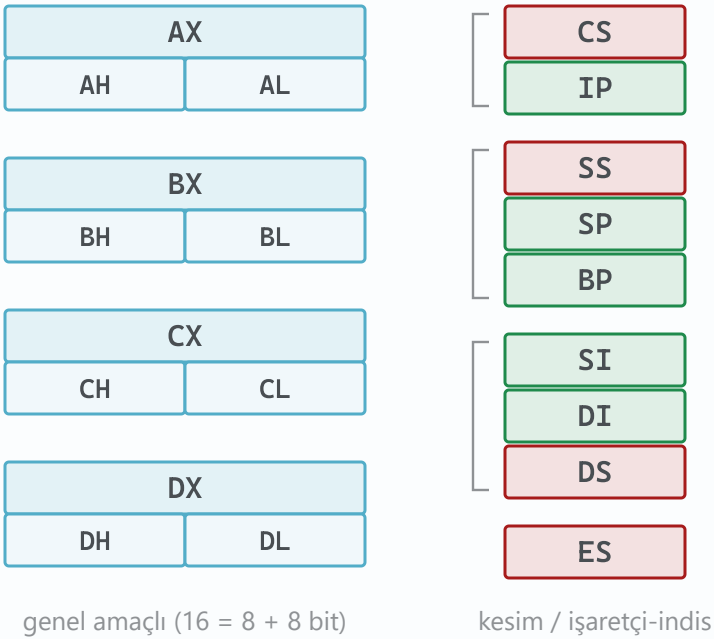


Yürüt: İşlem yapılır, sonuç yazılır, bayraklar güncellenir. Ardından döngü bir sonraki komutla tekrar başlar.

Adım 3 / 3

8086'da **BIU** (yol arayüz birimi) komutları önden getirip 6 baytlık **komut kuyruğuna** koyar; **EU** (yürütme birimi) bu sırada önceki komutu yürütür. Bu basit bir boru hattıdır (pipelining).

CPU'nun İçinde



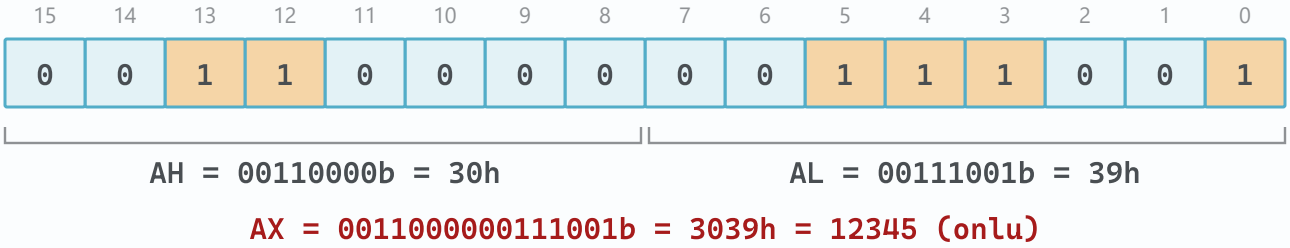
genel amaçlı (16 = 8 + 8 bit)

kesim / işaretçi-indis

Mavi: genel amaçlı · kırmızı: kesim yazmaçları · yeşil: işaretçi/indis yazmaçları. Köşeli çizgiler birlikte çalışan çiftleri gösterir (CS:IP, SS:SP/BP, DS:SI/DI).

Genel Amaçlı Yazmaçlar

- 8086 CPU'da **8 adet** genel amaçlı yazmaç vardır: AX, BX, CX, DX, SI, DI, BP, SP.
- Her yazmacın bir adı ve tipik bir görevi vardır; ana amaç bir sayıyı (değişkeni) **geçici olarak saklamaktır**.
- Hepsi **16 bit**tir. AX, BX, CX, DX ikişer adet **8 bitlik** yazmaçtan oluşur (üst: H, alt: L).
- CPU'nun içinde bulundukları için bellekten çok **daha hızlıdır**.



GENEL AMAÇLI YAZMAÇLAR

Yazmaçların Görevleri

Yazmaç	Adı	Tipik kullanımı
`AX` (AH, AL)	Akümülatör (Accumulator)	Aritmetik; MUL/DIV'in örtük işleneni; IN/OUT ile port verisi
`BX` (BH, BL)	Taban (Base)	Bellek adreslemede taban adres: `[BX]`
`CX` (CH, CL)	Sayaç (Count)	LOOP ve REP için sayaç; kaydırmalarda sayı CL'de
`DX` (DH, DL)	Veri (Data)	MUL/DIV'de sonucun üst yarısı; G/Ç port adresi
`SI`	Kaynak indis (Source Index)	Dizi/dizgi işlemlerinde kaynak adresi
`DI`	Hedef indis (Destination Index)	Dizi/dizgi işlemlerinde hedef adresi
`BP`	Taban işaretçisi (Base Pointer)	Yığındaki parametre ve yerel değişkenlere erişim
`SP`	Yığın işaretçisi (Stack Pointer)	Yığının tepesi; PUSH/POP/CALL/RET değiştirir

SI, DI, BP, SP'nin 8 bitlik yarıları yoktur. SP'yi genel amaçlı değişken gibi kullanmak yığını bozar.

İşlenen Türleri ve Örtük İşlenenler

Yazmaç
`MOV AX, BX` : değer işlemcinin içinden gelir; en hızlı tür.

Anlık değer
`MOV AX, 5` : değer komutun baytlarının içindedir; yalnızca kaynak olabilir.

Bellek
`MOV AX, [BX]` : değer bellekten bir yol döngüsüyle okunur.

Komut	Yazılan işlenen	Örtük işlenen(ler)
`MUL BL`	BL	AL (çarpılan), AX (sonuç)
`DIV BL`	BL	AX (bölünen), AL (bölüm), AH (kalan)
`LOOP don`	don	CX (sayaç)
`CBW`	—	AL → AX
`PUSH AX`	AX	SS:SP (yığın tepesi)
`XLAT`	—	AL (indis ve sonuç), BX (tablo adresi)

Örtük işlenen komutta yazılmaz ama değişir: `MUL BL` AH'yi de siler. Bir yazmacın değerini korumak isteyen program, kullandığı komutların örtük işlenenlerini bilmelidir.

Kod İzleme: MUL, SHR ve LOOP'un Örtük İşlenenleri

```
1  ORG 100h
▶ MOV AL, 12
▶ MOV BL, 10
4  MUL BL          ; AX = AL × BL
5  MOV CL, 2
6  SHR AX, CL      ; sayı CL'de
7  MOV CX, 3       ; LOOP sayacı
8  don:
9  ADD AX, 1
10 LOOP don        ; CX azalır
11 RET
```

AX	BX	CX	CF	ZF
000Ch	000Ah	0000h	0	0

AX = 12 (onluk)

AL = 12, BL = 10. AH başlangıçta 0.

Adım 1 / 8

DEĞİŞKENLER

AX = 000Ch | BX = 000Ah | CX = 0000h

AX, CL ve CX bu programda komutların “gizli” işlenenleriydi; yazmaçların görevleri tablosundaki roller buradan gelir.

Kod İzleme: MUL, SHR ve LOOP'un Örtük İşlenenleri

```
1  ORG 100h
2  MOV AL, 12
3  MOV BL, 10
▶ 4  MUL BL          ; AX = AL × BL
5  MOV CL, 2
6  SHR AX, CL        ; sayı CL'de
7  MOV CX, 3         ; LOOP sayacı
8  don:
9  ADD AX, 1
10 LOOP don          ; CX azalır
11 RET
```



`AX = 120 (onluk)`

örtük: AL, AX

`MUL BL` yalnız BL'yi yazar; çarpılan örtük olarak AL, sonuç örtük olarak AX'tir: AX = 120 = **0078h**. Sonuç 8 bite sığdığı için CF = 0.

Adım 2 / 8

DEĞİŞKENLER

AX = 0078h | BX = 000Ah | CX = 0000h

AX, CL ve CX bu programda komutların “gizli” işlenenleriydi; yazmaçların görevleri tablosundaki roller buradan gelir.

Kod İzleme: MUL, SHR ve LOOP'un Örtük İşlenenleri

```
1  ORG 100h
2  MOV AL, 12
3  MOV BL, 10
4  MUL BL          ; AX = AL × BL
▶  MOV CL, 2
▶  SHR AX, CL      ; sayı CL'de
7  MOV CX, 3       ; LOOP sayacı
8  don:
9  ADD AX, 1
10 LOOP don        ; CX azalır
11 RET
```

AX	BX	CX	CF	ZF
001Eh	000Ah	0002h	0	0

`AX = 30 (onluk)`

sayı CL'de

`SHR AX, CL` : 1'den büyük kaydırma sayısı yalnızca CL'de verilebilir. 120 ÷ 4 = 30 → AX = **001Eh**.

Adım 3 / 8

DEĞİŞKENLER

`AX = 001Eh` `BX = 000Ah` `CX = 0002h`

AX, CL ve CX bu programda komutların “gizli” işlenenleriydi; yazmaçların görevleri tablosundaki roller buradan gelir.

Kod İzleme: MUL, SHR ve LOOP'un Örtük İşlenenleri

```
1  ORG 100h
2  MOV AL, 12
3  MOV BL, 10
4  MUL BL          ; AX = AL × BL
5  MOV CL, 2
6  SHR AX, CL      ; sayı CL'de
▶  MOV CX, 3       ; LOOP sayacı
8  don:
9  ADD AX, 1
10 LOOP don        ; CX azalır
11 RET
```

AX	BX	CX	CF	ZF
001Eh	000Ah	0003h	0	0

AX = 30 (onluk)

CX = 3: LOOP'un örtük sayacı. CL de CX'in parçası olduğundan önceki 2 değeri silindi.

Adım 4 / 8

DEĞİŞKENLER

AX = 001Eh | BX = 000Ah | CX = 0003h

AX, CL ve CX bu programda komutların “gizli” işlenenleriydi; yazmaçların görevleri tablosundaki roller buradan gelir.

Kod İzleme: MUL, SHR ve LOOP'un Örtük İşlenenleri

```
1  ORG 100h
2  MOV AL, 12
3  MOV BL, 10
4  MUL BL          ; AX = AL × BL
5  MOV CL, 2
6  SHR AX, CL      ; sayı CL'de
7  MOV CX, 3       ; LOOP sayacı
8  don:
▶  ADD AX, 1
▶  LOOP don        ; CX azalır
11 RET
```

AX	BX	CX	CF	ZF
001Fh	000Ah	0002h	0	0

`AX = 31 (onluk)`

1. tur: AX + 1 = 31; `LOOP` CX'i 1 azaltır: CX = 2, sıfır değil → `don`.

Adım 5 / 8

DEĞİŞKENLER

`AX = 001Fh` `BX = 000Ah` `CX = 0002h`

AX, CL ve CX bu programda komutların “gizli” işlenenleriydi; yazmaçların görevleri tablosundaki roller buradan gelir.

Kod İzleme: MUL, SHR ve LOOP'un Örtük İşlenenleri

```
1  ORG 100h
2  MOV AL, 12
3  MOV BL, 10
4  MUL BL          ; AX = AL × BL
5  MOV CL, 2
6  SHR AX, CL      ; sayı CL'de
7  MOV CX, 3       ; LOOP sayacı
8  don:
▶  ADD AX, 1
▶  LOOP don        ; CX azalır
11 RET
```

AX	BX	CX	CF	ZF
0020h	000Ah	0001h	0	0

AX = 32 (onluk)

2. tur: AX + 1 = 32; `LOOP` CX'i 1 azaltır: CX = 1, sıfır değil → `don`.

Adım 6 / 8

DEĞİŞKENLER

AX = 0020h | BX = 000Ah | CX = 0001h

AX, CL ve CX bu programda komutların “gizli” işlenenleriydi; yazmaçların görevleri tablosundaki roller buradan gelir.

Kod İzleme: MUL, SHR ve LOOP'un Örtük İşlenenleri

```
1  ORG 100h
2  MOV AL, 12
3  MOV BL, 10
4  MUL BL          ; AX = AL × BL
5  MOV CL, 2
6  SHR AX, CL      ; sayı CL'de
7  MOV CX, 3       ; LOOP sayacı
8  don:
▶  ADD AX, 1
▶  LOOP don        ; CX azalır
11 RET
```

AX	BX	CX	CF	ZF
0021h	000Ah	0000h	0	0

AX = 33 (onluk)

3. tur: AX + 1 = 33; `LOOP` CX'i 1 azaltır: CX = 0, döngü biter.

Adım 7 / 8

DEĞİŞKENLER

AX = 0021h | BX = 000Ah | CX = 0000h

AX, CL ve CX bu programda komutların “gizli” işlenenleriydi; yazmaçların görevleri tablosundaki roller buradan gelir.

Kod İzleme: MUL, SHR ve LOOP'un Örtük İşlenenleri

```
1  ORG 100h
2  MOV AL, 12
3  MOV BL, 10
4  MUL BL          ; AX = AL × BL
5  MOV CL, 2
6  SHR AX, CL      ; sayı CL'de
7  MOV CX, 3       ; LOOP sayacı
8  don:
9  ADD AX, 1
10 LOOP don        ; CX azalır
▶  RET
```

AX	BX	CX	CF	ZF
0021h	000Ah	0000h	0	0

AX = 33 (onluk)

RET. Programda CX hiç azaltılmadı gibi görünse de LOOP onu üç kez değiştirdi: AX = **0021h** = 33.

Adım 8 / 8

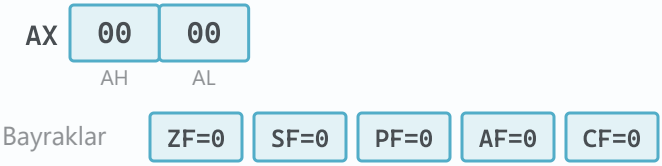
DEĞİŞKENLER

AX = 0021h | BX = 000Ah | CX = 0000h

AX, CL ve CX bu programda komutların “gizli” işlenenleriydi; yazmaçların görevleri tablosundaki roller buradan gelir.

Kod İzleme: AX, AH ve AL

```
1 ▶ ORG 100h
2   MOV AX, 3039h    ; AX = 3039h
3   MOV AL, 0FFh     ; yalnızca AL
4   MOV AH, 12h      ; yalnızca AH
5   INC AL           ; FFh + 1
6   RET
```



Başlangıç: AX = 0000h. `ORG 100h` makine koduna çevrilmez.

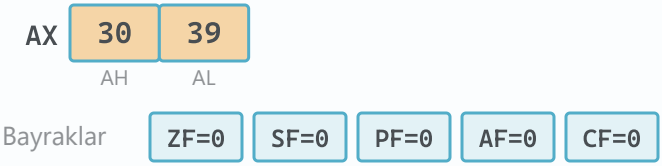
Adım 1 / 5

DEĞİŞKENLER

AX = 0000h | AH = 00h | AL = 00h | ZF = 0 | SF = 0 | PF = 0 | AF = 0 | CF = 0

Kod İzleme: AX, AH ve AL

```
1 ORG 100h
▶ MOV AX, 3039h    ; AX = 3039h
3 MOV AL, 0FFh     ; yalnızca AL
4 MOV AH, 12h      ; yalnızca AH
5 INC AL           ; FFh + 1
6 RET
```



`MOV AX, 3039h` : 16 bitin tamamı yazılır → AH = 30h, AL = 39h.

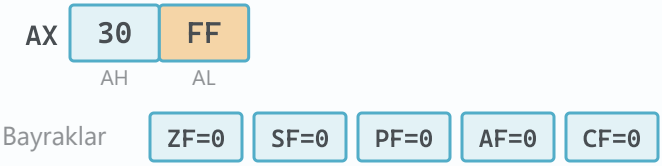
Adım 2 / 5

DEĞİŞKENLER

AX = 3039h | AH = 30h | AL = 39h | ZF = 0 | SF = 0 | PF = 0 | AF = 0 | CF = 0

Kod İzleme: AX, AH ve AL

```
1 ORG 100h
2 MOV AX, 3039h    ; AX = 3039h
▶ MOV AL, 0FFh     ; yalnızca AL
4 MOV AH, 12h      ; yalnızca AH
5 INC AL           ; FFh + 1
6 RET
```



`MOV AL, 0FFh`: yalnızca düşük bayt değişir → AX = **30FFh**.

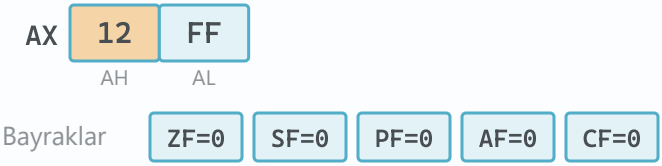
Adım 3 / 5

DEĞİŞKENLER

AX = **30FFh** | AH = **30h** | AL = **FFh** | ZF = **0** | SF = **0** | PF = **0** | AF = **0** | CF = **0**

Kod İzleme: AX, AH ve AL

```
1 ORG 100h
2 MOV AX, 3039h    ; AX = 3039h
3 MOV AL, 0FFh     ; yalnızca AL
▶ MOV AH, 12h      ; yalnızca AH
5 INC AL           ; FFh + 1
6 RET
```



`MOV AH, 12h`: yalnızca yüksek bayt değişir → AX = **12FFh**. MOV bayrakları etkilemez.

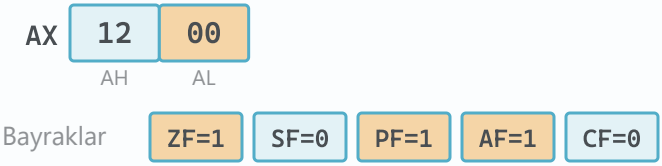
Adım 4 / 5

DEĞİŞKENLER

AX = **12FFh** | AH = **12h** | AL = **FFh** | ZF = **0** | SF = **0** | PF = **0** | AF = **0** | CF = **0**

Kod İzleme: AX, AH ve AL

```
1 ORG 100h
2 MOV AX, 3039h    ; AX = 3039h
3 MOV AL, 0FFh     ; yalnızca AL
4 MOV AH, 12h      ; yalnızca AH
▶ INC AL           ; FFh + 1
6 RET
```



`INC AL` : FFh + 1 = 00h (8 bit). Elde **AH'ye geçmez**: AX = **1200h**. ZF = 1; INC, CF'yi değiştirmez.

Adım 5 / 5

DEĞİŞKENLER

AX = 1200h | AH = 12h | AL = 00h | ZF = 1 | SF = 0 | PF = 1 | AF = 1 | CF = 0

GENEL AMAÇLI YAZMAÇLAR

AX Yazmacı Oyun Alanı

AX'e bir değer yazın; yüksek ve düşük baytları, ikili gösterimi ve onluk karşılıkları görün.

- AH ve AL ayrı yazmaçlar gibi kullanılabilir, ama **aynı** 16 bitin parçalarıdır.
- İşaretli yorumda bit 15 işaret bitidir (ikiye tümleyen).
- BX/BH/BL, CX/CH/CL, DX/DH/DL aynı şekilde çalışır.

AX = 3039 h

15	14	13	12	11	10	9	8
0	0	1	1	0	0	0	0

7	6	5	4	3	2	1	0
0	0	1	1	1	0	0	1

AX = **3039h** · AH = **30h** · AL = **39h**

işaretsiz 12345 · işaretli 12345

Turuncu: AH (bit 15–8) · mavi: AL (bit 7–0)

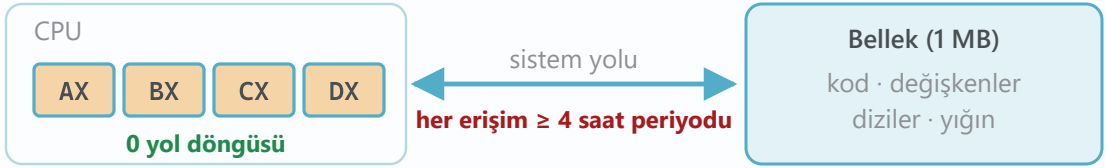
Yazmaç mı, Bellek mi?

Yazmaç

- CPU'nun içindedir; erişim için yol döngüsü gerekmez.
- Sayısı çok azdır (8086'da 14 adet 16 bitlik yazmaç).
- Ara sonuçlar ve sayaçlar için idealdir.

Bellek

- CPU'nun dışındadır; her erişim en az bir yol döngüsü (8086'da 4 saat periyodu) sürer.
- 1 MB'a kadar büyüktür.
- Program kodu, değişkenler, diziler ve yığın buradadır.



İyi assembly kodu sık kullanılan değerleri **yazmaçlarda** tutar, belleğe yalnızca gerektiğinde gider. Örneğin bir döngü sayacını bellekte değil `CX` 'te saklamak döngüyü hızlandırır.

14 yazmaç: 8 genel amaçlı + 4 kesim + IP + FLAGS.

Komut Süreleri: Yazmaç ve Bellek

Komut (8086)	Saat turu
`MOV AX, BX`	2
`MOV AX, 5`	4
`MOV AX, sayi`	10
`MOV CX, [BX]`	8 + 5 = 13
`ADD AX, BX`	3
`ADD AX, sayi`	9 + 6 = 15
`INC CX`	2
`INC sayac` (sözcük)	15 + 6 = 21

- Bellek işlenenli komutlarda süreye **etkin adres hesabı** (EA) eklenir: doğrudan adres 6, `[BX]` 5, `[BX+SI]` 7 tur.
- Belleğe yazan komutlar daha uzundur: `INC sayac` önce okur, sonra yazar (iki yol döngüsü).
- AX'e doğrudan adresten yükleme kısa akümülatör biçimini kullanır: 10 tur.
- Sayaç CX'te: 1000 tur × 2 = 2000 saat turu. Sayaç bellekte: 1000 × 21 = 21 000 tur; 5 MHz'de 0,4 ms yerine 4,2 ms.

Değerler Intel 8086 veri sayfasındaki en iyi durum süreleridir; tek adresteki sözcük erişimi ve önden getirme kuyruğunun boşalması süreyi uzatır. Oran değişmez: bellek işlenenli bir komut yazmaç işlenenli eşinden birkaç kat yavaştır.

Kesim Yazmaçları

Bellekteki erişilebilir **blokları** (kesim, segment) işaret etmek için kullanılır. Dört kesim yazmacı vardır:

CS · Code Segment
Geçerli programın (komutların) bulunduğu kesim. IP ile birlikte çalışır.

DS · Data Segment
Değişkenlerin tanımlandığı kesim. Bellek işlenenlerinin varsayılanı.

ES · Extra Segment
Kullanımı programcıya bağlı ek kesim; dizgi komutlarında hedef (ES:DI).

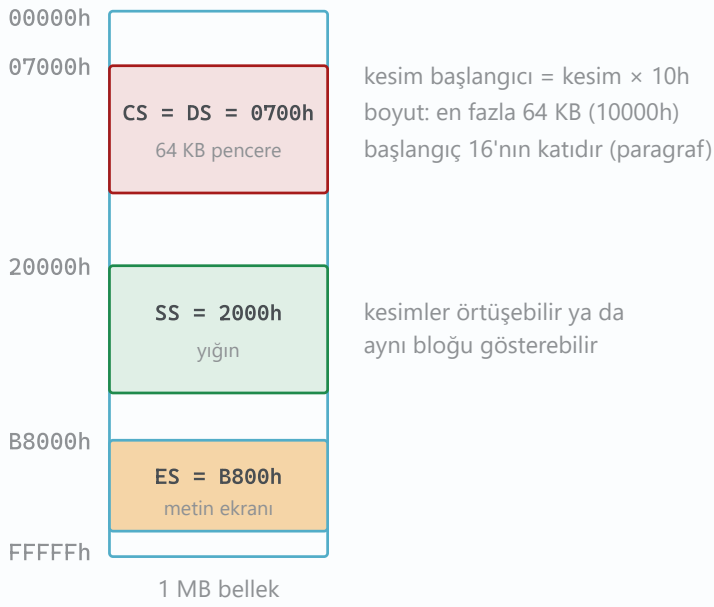
SS · Stack Segment
Yığını içeren kesim. SP ve BP ile birlikte çalışır.



Neden kesim? 8086'nın yazmaçları 16 bittir (en fazla 64 KB adresler) ama adres yolu 20 bittir (1 MB). İki 16 bitlik değer birleştirilerek 20 bitlik adres elde edilir.

KESİM YAZMAÇLARI

1 MB Bellek ve 64 KB Kesimler

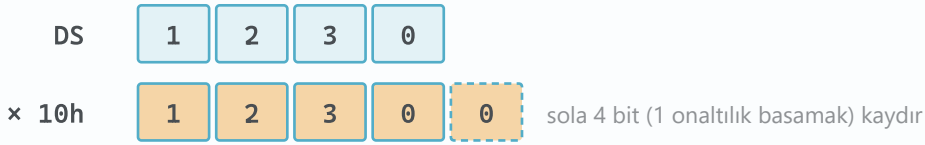


Fiziksel Adres Hesabı



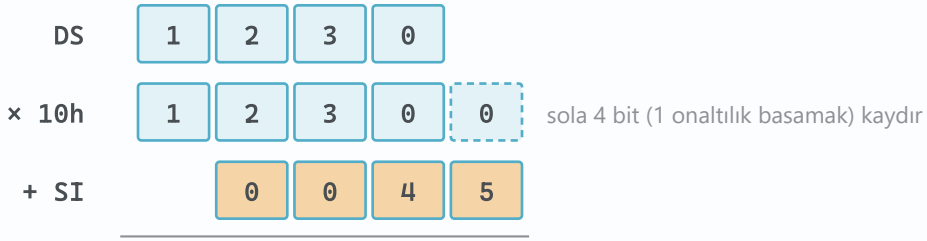
Fiziksel adresi **12345h** olan bayta erişmek için örneğin **DS = 1230h** ve **SI = 0045h** atanır.

Fiziksel Adres Hesabı



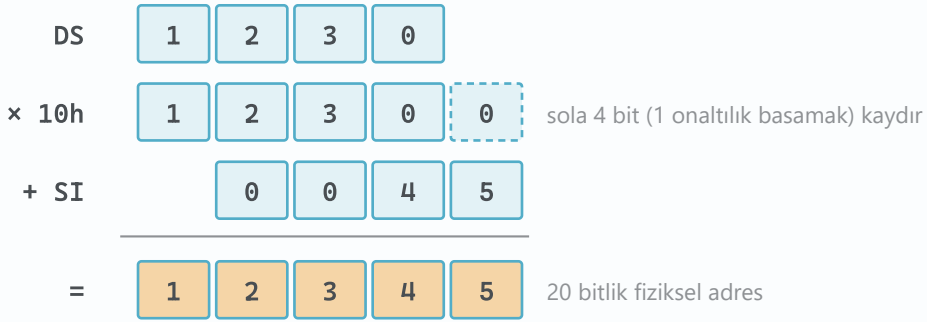
Kesim değeri **10h (16) ile çarpılır**: onaltılıkta sağa bir 0 eklemek, ikilide 4 bit sola kaydırmaktır → 12300h.

Fiziksel Adres Hesabı



Bağıl konum (offset) eklenmek üzere hizalanır: **0045h**.

Fiziksel Adres Hesabı



12300h + 0045h = 12345h. Gösterim: `1230:0045` (kesim:bağıl konum).

KESİM YAZMAÇLARI

Aynı Adres, Farklı Çiftler

Kesim:Bağıl konum	Hesap	Fiziksel
`1230:0045`	`12300 + 0045`	`12345h`
`1234:0005`	`12340 + 0005`	`12345h`
`1200:0345`	`12000 + 0345`	`12345h`
`1000:2345`	`10000 + 2345`	`12345h`
`0235:FFF5`	`02350 + FFF5`	`12345h`

- Kesimler 16 baytta bir başlar, 64 KB uzunluktadır → **örtüşürler**.
- Bu nedenle bir fiziksel adrese 4096'ya kadar farklı kesim:bağıl konum çifti karşılık gelebilir.
- **Normalize** biçimde bağıl konum 0–Fh arasıdır: `1234:0005`.
- Toplam 20 biti aşarsa 8086'da adres **başa sarar**: `FFFF:0010` → 00000h.

Kod İzleme: Üç Çift, Tek Bayt

```
1  ORG 100h
▶ MOV AX, 1230h
3  MOV DS, AX
4  MOV SI, 0045h
5  MOV BYTE PTR [SI], 7Ah
6  MOV AX, 1234h
7  MOV DS, AX
8  MOV BX, 0005h
9  MOV CL, [BX]     ; 1234:0005
10 MOV AX, 1000h
11 MOV ES, AX
12 MOV DI, 2345h
13 MOV DL, ES:[DI]  ; 1000:2345
14 RET
```

fiziksel adres = kesim × 10h + bağıl konum

DS × 10h	?????h
+ SI	????h
= fiziksel	?

bellek: 12345h

00h

bulunan çiftler

AX = 1230h. Kesim yazmacına sabit doğrudan yazılamadığı için değer önce AX'e alınır.

Adım 1 / 12

DEĞİŞKENLER

AX = 1230h | DS = 0700h | ES = 0700h | SI = 0000h

Kesimler 16 baytta bir başlayıp örtüştüğü için aynı fiziksel bayta pek çok kesim:bağıl konum çiftiyle ulaşılır.

Kod İzleme: Üç Çift, Tek Bayt

```
1  ORG 100h
2  MOV AX, 1230h
▶  MOV DS, AX
4  MOV SI, 0045h
5  MOV BYTE PTR [SI], 7Ah
6  MOV AX, 1234h
7  MOV DS, AX
8  MOV BX, 0005h
9  MOV CL, [BX]     ; 1234:0005
10 MOV AX, 1000h
11 MOV ES, AX
12 MOV DI, 2345h
13 MOV DL, ES:[DI]  ; 1000:2345
14 RET
```

fiziksel adres = kesim × 10h + bağıl konum

DS × 10h	12300h
+ SI	????h
= fiziksel	?

bellek: 12345h

00h

bulunan çiftler

DS = **1230h**: kesim 12300h fiziksel adresinden başlar.

Adım 2 / 12

DEĞİŞKENLER

AX = 1230h | DS = 1230h | ES = 0700h | SI = 0000h

Kesimler 16 baytta bir başlayıp örtüştüğü için aynı fiziksel bayta pek çok kesim:bağıl konum çiftiyle ulaşılır.

Kod İzleme: Üç Çift, Tek Bayt

```
1  ORG 100h
2  MOV AX, 1230h
3  MOV DS, AX
▶  MOV SI, 0045h
5  MOV BYTE PTR [SI], 7Ah
6  MOV AX, 1234h
7  MOV DS, AX
8  MOV BX, 0005h
9  MOV CL, [BX]     ; 1234:0005
10 MOV AX, 1000h
11 MOV ES, AX
12 MOV DI, 2345h
13 MOV DL, ES:[DI]  ; 1000:2345
14 RET
```

fiziksel adres = kesim × 10h + bağıl konum

DS × 10h	12300h
+ SI	0045h
= fiziksel	12345h

bulunan çiftler

bellek: 12345h

00h

SI = 0045h: kesim içindeki bağıl konum.

Adım 3 / 12

DEĞİŞKENLER

AX = 1230h | DS = 1230h | ES = 0700h | SI = 0045h

Kesimler 16 baytta bir başlayıp örtüştüğü için aynı fiziksel bayta pek çok kesim:bağıl konum çiftiyle ulaşılır.

Kod İzleme: Üç Çift, Tek Bayt

```
1  ORG 100h
2  MOV AX, 1230h
3  MOV DS, AX
4  MOV SI, 0045h
▶  MOV BYTE PTR [SI], 7Ah
6  MOV AX, 1234h
7  MOV DS, AX
8  MOV BX, 0005h
9  MOV CL, [BX]      ; 1234:0005
10 MOV AX, 1000h
11 MOV ES, AX
12 MOV DI, 2345h
13 MOV DL, ES:[DI]   ; 1000:2345
14 RET
```

fiziksel adres = kesim × 10h + bağıl konum

DS × 10h	12300h
+ SI	0045h
= fiziksel	12345h

bellek: 12345h

7Ah

bulunan çiftler

1230:0045
→ 12345h

1230h × 10h + 0045h = **12345h**. Bu bayta **7Ah** yazıldı.

Adım 4 / 12

DEĞİŞKENLER

AX = 1230h | DS = 1230h | ES = 0700h | SI = 0045h

Kesimler 16 baytta bir başlayıp örtüştüğü için aynı fiziksel bayta pek çok kesim:bağıl konum çiftiyle ulaşılır.

Kod İzleme: Üç Çift, Tek Bayt

```
1  ORG 100h
2  MOV AX, 1230h
3  MOV DS, AX
4  MOV SI, 0045h
5  MOV BYTE PTR [SI], 7Ah
▶  MOV AX, 1234h
7  MOV DS, AX
8  MOV BX, 0005h
9  MOV CL, [BX]      ; 1234:0005
10 MOV AX, 1000h
11 MOV ES, AX
12 MOV DI, 2345h
13 MOV DL, ES:[DI]   ; 1000:2345
14 RET
```

fiziksel adres = kesim × 10h + bağıl konum

DS × 10h	?????h
+ BX	????h
= fiziksel	?

bellek: 12345h

7Ah

bulunan çiftler

1230:0045
→ 12345h

AX = 1234h: ikinci kesim değeri.

Adım 5 / 12

DEĞİŞKENLER

AX = 1234h | DS = 1230h | ES = 0700h | BX = 0000h

Kesimler 16 baytta bir başlayıp örtüştüğü için aynı fiziksel bayta pek çok kesim:bağıl konum çiftiyle ulaşılır.

Kod İzleme: Üç Çift, Tek Bayt

```
1  ORG 100h
2  MOV AX, 1230h
3  MOV DS, AX
4  MOV SI, 0045h
5  MOV BYTE PTR [SI], 7Ah
6  MOV AX, 1234h
▶  MOV DS, AX
8  MOV BX, 0005h
9  MOV CL, [BX]     ; 1234:0005
10 MOV AX, 1000h
11 MOV ES, AX
12 MOV DI, 2345h
13 MOV DL, ES:[DI]  ; 1000:2345
14 RET
```

fiziksel adres = kesim × 10h + bağıl konum

DS × 10h	12340h
+ BX	????h
= fiziksel	?

bellek: 12345h

7Ah

bulunan çiftler

1230:0045
→ 12345h

DS = **1234h**: kesim 12340h'den başlar; öncekinden yalnızca 4 bayt ileride.

Adım 6 / 12

DEĞİŞKENLER

AX = 1234h | DS = 1234h | ES = 0700h | BX = 0000h

Kesimler 16 baytta bir başlayıp örtüştüğü için aynı fiziksel bayta pek çok kesim:bağıl konum çiftiyle ulaşılır.

Kod İzleme: Üç Çift, Tek Bayt

```
1  ORG 100h
2  MOV AX, 1230h
3  MOV DS, AX
4  MOV SI, 0045h
5  MOV BYTE PTR [SI], 7Ah
6  MOV AX, 1234h
7  MOV DS, AX
▶  MOV BX, 0005h
9  MOV CL, [BX]      ; 1234:0005
10 MOV AX, 1000h
11 MOV ES, AX
12 MOV DI, 2345h
13 MOV DL, ES:[DI]   ; 1000:2345
14 RET
```

fiziksel adres = kesim × 10h + bağıl konum

DS × 10h	12340h
+ BX	0005h
= fiziksel	12345h

bellek: 12345h

7Ah

bulunan çiftler

1230:0045
→ 12345h

BX = 0005h.

Adım 7 / 12

DEĞİŞKENLER

AX = 1234h | DS = 1234h | ES = 0700h | BX = 0005h

Kesimler 16 baytta bir başlayıp örtüştüğü için aynı fiziksel bayta pek çok kesim:bağıl konum çiftiyle ulaşılır.

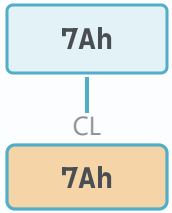
Kod İzleme: Üç Çift, Tek Bayt

```
1  ORG 100h
2  MOV AX, 1230h
3  MOV DS, AX
4  MOV SI, 0045h
5  MOV BYTE PTR [SI], 7Ah
6  MOV AX, 1234h
7  MOV DS, AX
8  MOV BX, 0005h
▶  MOV CL, [BX]      ; 1234:0005
10 MOV AX, 1000h
11 MOV ES, AX
12 MOV DI, 2345h
13 MOV DL, ES:[DI]   ; 1000:2345
14 RET
```

fiziksel adres = kesim × 10h + bağıl konum

DS × 10h	12340h
+ BX	0005h
= fiziksel	12345h

bellek: 12345h



bulunan çiftler

1230:0045 → 12345h

1234:0005 → 12345h

1234h × 10h + 0005h = **12345h**: aynı bayt! CL = **7Ah**.

Adım 8 / 12

DEĞİŞKENLER

AX = 1234h | DS = 1234h | ES = 0700h | BX = 0005h

Kesimler 16 baytta bir başlayıp örtüştüğü için aynı fiziksel bayta pek çok kesim:bağıl konum çiftiyle ulaşılır.

Kod İzleme: Üç Çift, Tek Bayt

```
1  ORG 100h
2  MOV AX, 1230h
3  MOV DS, AX
4  MOV SI, 0045h
5  MOV BYTE PTR [SI], 7Ah
6  MOV AX, 1234h
7  MOV DS, AX
8  MOV BX, 0005h
9  MOV CL, [BX]      ; 1234:0005
▶  MOV AX, 1000h
11 MOV ES, AX
12 MOV DI, 2345h
13 MOV DL, ES:[DI]   ; 1000:2345
14 RET
```

fiziksel adres = kesim × 10h + bağıl konum

ES × 10h	?????h
+ DI	????h
= fiziksel	?

bellek: 12345h

7Ah

bulunan çiftler

1230:0045 → 12345h

1234:0005 → 12345h

AX = 1000h.

Adım 9 / 12

DEĞİŞKENLER

AX = 1000h | DS = 1234h | ES = 0700h | DI = 0000h

Kesimler 16 baytta bir başlayıp örtüştüğü için aynı fiziksel bayta pek çok kesim:bağıl konum çiftiyle ulaşılır.

Kod İzleme: Üç Çift, Tek Bayt

```
1  ORG 100h
2  MOV AX, 1230h
3  MOV DS, AX
4  MOV SI, 0045h
5  MOV BYTE PTR [SI], 7Ah
6  MOV AX, 1234h
7  MOV DS, AX
8  MOV BX, 0005h
9  MOV CL, [BX]      ; 1234:0005
10 MOV AX, 1000h
▶  MOV ES, AX
12 MOV DI, 2345h
13 MOV DL, ES:[DI]   ; 1000:2345
14 RET
```

fiziksel adres = kesim × 10h + bağıl konum

ES × 10h	10000h
+ DI	????h
= fiziksel	?

bellek: 12345h

7Ah

bulunan çiftler

1230:0045 → 12345h

1234:0005 → 12345h

ES = **1000h**. Bu kez DS'ye dokunmadan ek kesim kullanılacak.

Adım 10 / 12

DEĞİŞKENLER

AX = 1000h | DS = 1234h | ES = 1000h | DI = 0000h

Kesimler 16 baytta bir başlayıp örtüştüğü için aynı fiziksel bayta pek çok kesim:bağıl konum çiftiyle ulaşılır.

Kod İzleme: Üç Çift, Tek Bayt

```
1  ORG 100h
2  MOV AX, 1230h
3  MOV DS, AX
4  MOV SI, 0045h
5  MOV BYTE PTR [SI], 7Ah
6  MOV AX, 1234h
7  MOV DS, AX
8  MOV BX, 0005h
9  MOV CL, [BX]      ; 1234:0005
10 MOV AX, 1000h
11 MOV ES, AX
▶ MOV DI, 2345h
13 MOV DL, ES:[DI]   ; 1000:2345
14 RET
```

fiziksel adres = kesim × 10h + bağıl konum

ES × 10h	10000h
+ DI	2345h
= fiziksel	12345h

bellek: 12345h

7Ah

bulunan çiftler

1230:0045 → 12345h

1234:0005 → 12345h

DI = 2345h.

Adım 11 / 12

DEĞİŞKENLER

AX = 1000h | DS = 1234h | ES = 1000h | DI = 2345h

Kesimler 16 baytta bir başlayıp örtüştüğü için aynı fiziksel bayta pek çok kesim:bağıl konum çiftiyle ulaşılır.

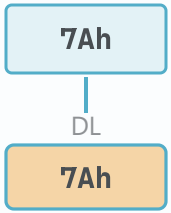
Kod İzleme: Üç Çift, Tek Bayt

```
1  ORG 100h
2  MOV AX, 1230h
3  MOV DS, AX
4  MOV SI, 0045h
5  MOV BYTE PTR [SI], 7Ah
6  MOV AX, 1234h
7  MOV DS, AX
8  MOV BX, 0005h
9  MOV CL, [BX]      ; 1234:0005
10 MOV AX, 1000h
11 MOV ES, AX
12 MOV DI, 2345h
▶ MOV DL, ES:[DI]   ; 1000:2345
14 RET
```

fiziksel adres = kesim × 10h + bağıl konum

ES × 10h	10000h
+ DI	2345h
= fiziksel	**12345h**

bellek: 12345h



bulunan çiftler

`1230:0045` → 12345h `1234:0005` → 12345h `1000:2345` → 12345h

`ES:[DI]` : 10000h + 2345h = **12345h** → DL = **7Ah**. Üç farklı kesim:bağıl konum çifti, tek fiziksel adres.

Adım 12 / 12

DEĞİŞKENLER

`AX = 1000h` `DS = 1234h` `ES = 1000h` `DI = 2345h`

Kesimler 16 baytta bir başlayıp örtüştüğü için aynı fiziksel bayta pek çok kesim:bağıl konum çiftiyle ulaşılır.

KESİM YAZMAÇLARI

Fiziksel Adres Hesaplayıcı

Kesim 1230 h : Bağıl konum 0045 h

1230h × 10h = 12300h

12300h + 0045h = **12345h**

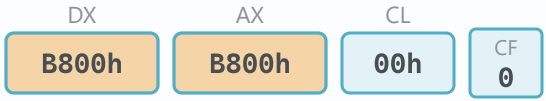
Normalize: **1234:0005**

B800:015E | 1234:7890 | 0700:0100 | FFFF:0010

Onaltılık en fazla 4 basamak girin. Hesap: fiziksel = kesim × 10h + bağıl konum (20 bit).

Kod İzleme: Fiziksel Adresi Yazılımla Hesaplama

```
1  ORG 100h
▶  MOV AX, kesim   ; B800h
▶  MOV DX, AX
4  MOV CL, 4
5  SHL AX, CL      ; düşük 16 bit
6  MOV CL, 12
7  SHR DX, CL      ; üst 4 bit
8  ADD AX, konum   ; + bağıl konum
9  ADC DX, 0       ; elde üst kısma
10 RET
11 kesim DW 0B800h
12 konum DW 9F3Ch
```



DX:AX = 0B800h

hedef: B800h × 10h + 9F3Ch = C1F3Ch

Kesim değeri B800h hem AX'e hem DX'e alınır: iki parça ayrı ayrı hazırlanacak.

Adım 1 / 6

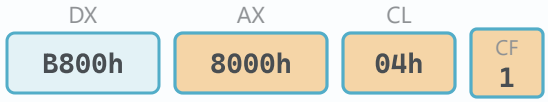
DEĞİŞKENLER

AX = **B800h** | DX = **B800h** | CL = **00h** | CF = **0**

20 bitlik adres 16 bitlik yazmaca sığmadığı için iki yazmaca bölündü. ADC olmasaydı elde kaybolur ve sonuç 01F3Ch gibi yanlış bir adres olurdu.

Kod İzleme: Fiziksel Adresi Yazılımla Hesaplama

```
1  ORG 100h
2  MOV AX, kesim    ; B800h
3  MOV DX, AX
▶  MOV CL, 4
▶  SHL AX, CL       ; düşük 16 bit
6  MOV CL, 12
7  SHR DX, CL       ; üst 4 bit
8  ADD AX, konum    ; + bağıl konum
9  ADC DX, 0        ; elde üst kısma
10 RET
11 kesim DW 0B800h
12 konum DW 9F3Ch
```



DX:AX = 08000h

hedef: B800h × 10h + 9F3Ch = C1F3Ch

`SHL AX, CL` (CL = 4): × 16 = B8000h, ama AX 16 bittir; düşük kısım **8000h** kalır, en üstteki B dışarı taşar.

Adım 2 / 6

DEĞİŞKENLER

AX = 8000h | DX = B800h | CL = 04h | CF = 1

20 bitlik adres 16 bitlik yazmaca sığmadığı için iki yazmaca bölündü. ADC olmasaydı elde kaybolur ve sonuç 01F3Ch gibi yanlış bir adres olurdu.

Kod İzleme: Fiziksel Adresi Yazılımla Hesaplama

```
1  ORG 100h
2  MOV AX, kesim   ; B800h
3  MOV DX, AX
4  MOV CL, 4
5  SHL AX, CL      ; düşük 16 bit
▶  MOV CL, 12
▶  SHR DX, CL      ; üst 4 bit
8  ADD AX, konum   ; + bağıl konum
9  ADC DX, 0       ; elde üst kısma
10 RET
11 kesim DW 0B800h
12 konum DW 9F3Ch
```

DX	AX	CL	CF
000Bh	8000h	0Ch	1

DX:AX = B8000h

hedef: B800h × 10h + 9F3Ch = C1F3Ch

`SHR DX, CL` (CL = 12): kesimin üst 4 biti DX'e: DX = **000Bh**. 20 bitlik değer DX:AX çiftinde.

Adım 3 / 6

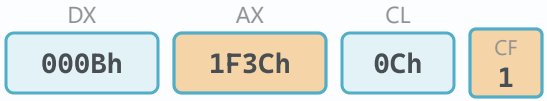
DEĞİŞKENLER

AX = 8000h | DX = 000Bh | CL = 0Ch | CF = 1

20 bitlik adres 16 bitlik yazmaca sığmadığı için iki yazmaca bölündü. ADC olmasaydı elde kaybolur ve sonuç 01F3Ch gibi yanlış bir adres olurdu.

Kod İzleme: Fiziksel Adresi Yazılımla Hesaplama

```
1  ORG 100h
2  MOV AX, kesim    ; B800h
3  MOV DX, AX
4  MOV CL, 4
5  SHL AX, CL       ; düşük 16 bit
6  MOV CL, 12
7  SHR DX, CL       ; üst 4 bit
▶  ADD AX, konum    ; + bağıl konum
9  ADC DX, 0        ; elde üst kısma
10 RET
11 kesim DW 0B800h
12 konum DW 9F3Ch
```



DX:AX = B1F3Ch — **elde: CF = 1**

hedef: B800h × 10h + 9F3Ch = C1F3Ch

`ADD AX, konum` : 8000h + 9F3Ch = 11F3Ch → AX = **1F3Ch**, CF = 1. Toplama 16 biti aştı.

Adım 4 / 6

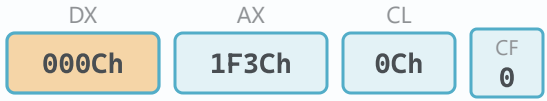
DEĞİŞKENLER

AX = **1F3Ch** | DX = **000Bh** | CL = **0Ch** | CF = **1**

20 bitlik adres 16 bitlik yazmaca sığmadığı için iki yazmaca bölündü. ADC olmasaydı elde kaybolur ve sonuç 01F3Ch gibi yanlış bir adres olurdu.

Kod İzleme: Fiziksel Adresi Yazılımla Hesaplama

```
1  ORG 100h
2  MOV AX, kesim   ; B800h
3  MOV DX, AX
4  MOV CL, 4
5  SHL AX, CL      ; düşük 16 bit
6  MOV CL, 12
7  SHR DX, CL      ; üst 4 bit
8  ADD AX, konum   ; + bağıl konum
▶  ADC DX, 0       ; elde üst kısma
10 RET
11 kesim DW 0B800h
12 konum DW 9F3Ch
```



DX:AX = C1F3Ch

hedef: B800h × 10h + 9F3Ch = C1F3Ch

`ADC DX, 0` eldeyi üst kısma ekler: DX = **000Ch**. DX:AX = **C1F3Ch**.

Adım 5 / 6

DEĞİŞKENLER

AX = **1F3Ch** | DX = **000Ch** | CL = **0Ch** | CF = **0**

20 bitlik adres 16 bitlik yazmaca sığmadığı için iki yazmaca bölündü. ADC olmasaydı elde kaybolur ve sonuç 01F3Ch gibi yanlış bir adres olurdu.

Kod İzleme: Fiziksel Adresi Yazılımla Hesaplama

```
1  ORG 100h
2  MOV AX, kesim    ; B800h
3  MOV DX, AX
4  MOV CL, 4
5  SHL AX, CL       ; düşük 16 bit
6  MOV CL, 12
7  SHR DX, CL       ; üst 4 bit
8  ADD AX, konum    ; + bağıl konum
9  ADC DX, 0        ; elde üst kısma
▶  RET
11 kesim DW 0B800h
12 konum DW 9F3Ch
```

DX	AX	CL	CF
000Ch	1F3Ch	0Ch	0

DX:AX = C1F3Ch

hedef: B800h × 10h + 9F3Ch = C1F3Ch

RET. Yazılım, BIU'daki adres toplayıcısının tek adımda yaptığı işi 7 komutla yaptı.

Adım 6 / 6

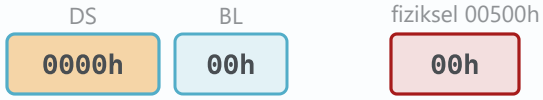
DEĞİŞKENLER

AX = 1F3Ch · DX = 000Ch · CL = 0Ch · CF = 0

20 bitlik adres 16 bitlik yazmaca sığmadığı için iki yazmaca bölündü. ADC olmasaydı elde kaybolur ve sonuç 01F3Ch gibi yanlış bir adres olurdu.

Kod İzleme: 1 MB Sınırında Adres Başa Sarar

```
1 ORG 100h
▶ MOV AX, 0
▶ MOV DS, AX
4 MOV BYTE PTR [0500h], 77h
5 MOV AX, 0FFFFh
6 MOV DS, AX
7 MOV BL, [0510h]    ; FFFF:0510
8 RET
```



DS = 0000h: kesim belleğin en başında. Kesim yazmacına sabit yazılamadığı için değer AX'ten geçer.

Adım 1 / 5

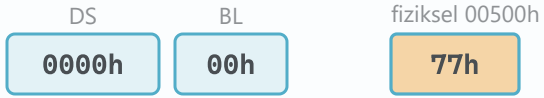
DEĞİŞKENLER

DS = 0000h BL = 00h

80286 ve sonrasında 21. adres hattı (A20) açıksa FFFF:0510 gerçekten 100500h'ye ulaşır; 8086 için yazılmış bazı eski programlar başa sarmaya güvendiği için bu hat kapatılabilir.

Kod İzleme: 1 MB Sınırında Adres Başa Sarar

```
1 ORG 100h
2 MOV AX, 0
3 MOV DS, AX
▶ MOV BYTE PTR [0500h], 77h
5 MOV AX, 0FFFFh
6 MOV DS, AX
7 MOV BL, [0510h]    ; FFFF:0510
8 RET
```



`0000h × 10h + 0500h`
= 00500h

`MOV BYTE PTR [0500h], 77h` : 0000:0500 → fiziksel **00500h** adresine 77h yazıldı.

Adım 2 / 5

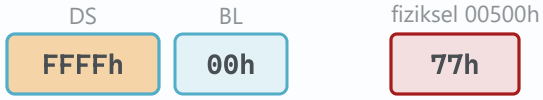
DEĞİŞKENLER

`DS = 0000h` `BL = 00h`

80286 ve sonrasında 21. adres hattı (A20) açıksa FFFF:0510 gerçekten 100500h'ye ulaşır; 8086 için yazılmış bazı eski programlar başa sarmaya güvendiği için bu hat kapatılabilir.

Kod İzleme: 1 MB Sınırında Adres Başa Sarar

```
1 ORG 100h
2 MOV AX, 0
3 MOV DS, AX
4 MOV BYTE PTR [0500h], 77h
▶ MOV AX, 0FFFFh
▶ MOV DS, AX
7 MOV BL, [0510h]    ; FFFF:0510
8 RET
```



DS = **FFFFh**: kesim FFFF0h'den, 1 MB'ın son 16 baytından başlar.

Adım 3 / 5

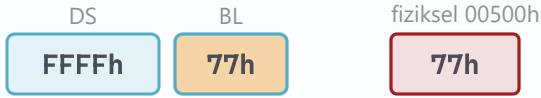
DEĞİŞKENLER

DS = FFFFh BL = 00h

80286 ve sonrasında 21. adres hattı (A20) açıksa FFFF:0510 gerçekten 100500h'ye ulaşır; 8086 için yazılmış bazı eski programlar başa sarmaya güvendiği için bu hat kapatılabilir.

Kod İzleme: 1 MB Sınırında Adres Başa Sarar

```
1 ORG 100h
2 MOV AX, 0
3 MOV DS, AX
4 MOV BYTE PTR [0500h], 77h
5 MOV AX, 0FFFFh
6 MOV DS, AX
▶ MOV BL, [0510h]    ; FFFF:0510
8 RET
```



`FFFF0h + 0510h = 100500h`

21. bit atılır → 00500h

başa sardı

`MOV BL, [0510h]` : toplam 21 bit olur; 8086'nın 20 adres hattı yalnızca 00500h'yi taşır. Aynı bayt okundu: BL = **77h**.

Adım 4 / 5

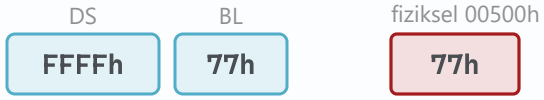
DEĞİŞKENLER

DS = **FFFFh** BL = **77h**

80286 ve sonrasında 21. adres hattı (A20) açıksa FFFF:0510 gerçekten 100500h'ye ulaşır; 8086 için yazılmış bazı eski programlar başa sarmaya güvendiği için bu hat kapatılabilir.

Kod İzleme: 1 MB Sınırında Adres Başa Sarar

```
1 ORG 100h
2 MOV AX, 0
3 MOV DS, AX
4 MOV BYTE PTR [0500h], 77h
5 MOV AX, 0FFFFh
6 MOV DS, AX
7 MOV BL, [0510h]    ; FFFF:0510
▶ RET
```



RET. FFFF:0010'dan FFFF:FFFF'e kadarki adresler belleğin başındaki 00000h–0FFEFh bölgesine düşer.

Adım 5 / 5

DEĞİŞKENLER

DS = **FFFFh** BL = **77h**

80286 ve sonrasında 21. adres hattı (A20) açıksa FFFF:0510 gerçekten 100500h'ye ulaşır; 8086 için yazılmış bazı eski programlar başa sarmaya güvendiği için bu hat kapatılabilir.

KESİM YAZMAÇLARI

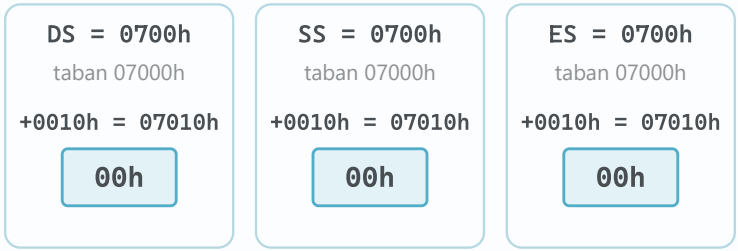
Varsayılan Kesim Eşleşmeleri

Bağıl konumu veren	Varsayılan kesim
`IP` (sıradaki komut)	`CS`
`SP` (yığın tepesi)	`SS`
`BP` içeren adresleme	`SS`
`BX`, `SI`, `DI`, sabit adres	`DS`
Dizgi komutlarında `DI`	`ES`

- Kesim ve genel amaçlı yazmaçlar birlikte kullanılarak bellekteki herhangi bir değere erişilir.
- Köşeli parantez içinde yazmaçlardan hesaplanan bağıl konuma **etkin adres** (EA) denir; fiziksel adres = kesim × 10h + EA.
- Adreslemede yalnızca **BX, SI, DI, BP** kullanılabilir; `[BX]` geçerli, `[BL]`, `[BH]`, `[AX]` geçersiz.
- Varsayılan değiştirilebilir: `MOV AL, ES:[BX]` (kesim önekli erişim).

Kod İzleme: [BX], [BP] ve ES:[BX]

```
1  ORG 100h
▶  MOV AX, 2000h
3  MOV DS, AX
4  MOV AX, 3000h
5  MOV ES, AX
6  MOV BX, 0010h
7  MOV BP, BX       ; aynı ofset
8  MOV BYTE PTR [BX], 11h
9  MOV BYTE PTR [BP], 22h
10 MOV BYTE PTR ES:[BX], 33h
11 RET
```



BX = 0000h BP = 0000h

AX = 2000h. .COM programında başlangıçta CS = DS = ES = SS = **0700h**.

Adım 1 / 9

DEĞİŞKENLER

DS = 0700h | ES = 0700h | SS = 0700h | BX = 0000h | BP = 0000h

Bağıl konum aynı olsa da kesim farklıysa erişilen bayt farklıdır: BP içeren adres SS, BX/SI/DI içeren adres DS kullanır.

Kod İzleme: [BX], [BP] ve ES:[BX]

```
1  ORG 100h
2  MOV AX, 2000h
▶  MOV DS, AX
4  MOV AX, 3000h
5  MOV ES, AX
6  MOV BX, 0010h
7  MOV BP, BX       ; aynı ofset
8  MOV BYTE PTR [BX], 11h
9  MOV BYTE PTR [BP], 22h
10 MOV BYTE PTR ES:[BX], 33h
11 RET
```



DS = **2000h**: veri kesimi artık 20000h'de.

Adım 2 / 9

DEĞİŞKENLER

DS = 2000h | ES = 0700h | SS = 0700h | BX = 0000h | BP = 0000h

Bağıl konum aynı olsa da kesim farklıysa erişilen bayt farklıdır: BP içeren adres SS, BX/SI/DI içeren adres DS kullanır.

Kod İzleme: [BX], [BP] ve ES:[BX]

```
1  ORG 100h
2  MOV AX, 2000h
3  MOV DS, AX
▶  MOV AX, 3000h
5  MOV ES, AX
6  MOV BX, 0010h
7  MOV BP, BX       ; aynı ofset
8  MOV BYTE PTR [BX], 11h
9  MOV BYTE PTR [BP], 22h
10 MOV BYTE PTR ES:[BX], 33h
11 RET
```

DS = 2000h	SS = 0700h	ES = 0700h
taban 20000h	taban 07000h	taban 07000h
+0010h = 20010h	+0010h = 07010h	+0010h = 07010h
00h	00h	00h

BX = 0000h BP = 0000h

AX = 3000h.

Adım 3 / 9

DEĞİŞKENLER

DS = 2000h | ES = 0700h | SS = 0700h | BX = 0000h | BP = 0000h

Bağıl konum aynı olsa da kesim farklıysa erişilen bayt farklıdır: BP içeren adres SS, BX/SI/DI içeren adres DS kullanır.

Kod İzleme: [BX], [BP] ve ES:[BX]

```
1  ORG 100h
2  MOV AX, 2000h
3  MOV DS, AX
4  MOV AX, 3000h
▶  MOV ES, AX
6  MOV BX, 0010h
7  MOV BP, BX       ; aynı ofset
8  MOV BYTE PTR [BX], 11h
9  MOV BYTE PTR [BP], 22h
10 MOV BYTE PTR ES:[BX], 33h
11 RET
```

DS = 2000h	SS = 0700h	ES = 3000h
taban 20000h	taban 07000h	taban 30000h
+0010h = 20010h	+0010h = 07010h	+0010h = 30010h
00h	00h	00h

BX = 0000h BP = 0000h

ES = **3000h**. SS değişmedi: 0700h.

Adım 4 / 9

DEĞİŞKENLER

DS = 2000h | ES = 3000h | SS = 0700h | BX = 0000h | BP = 0000h

Bağıl konum aynı olsa da kesim farklıysa erişilen bayt farklıdır: BP içeren adres SS, BX/SI/DI içeren adres DS kullanır.

Kod İzleme: [BX], [BP] ve ES:[BX]

```
1  ORG 100h
2  MOV AX, 2000h
3  MOV DS, AX
4  MOV AX, 3000h
5  MOV ES, AX
▶  MOV BX, 0010h
7  MOV BP, BX        ; aynı ofset
8  MOV BYTE PTR [BX], 11h
9  MOV BYTE PTR [BP], 22h
10 MOV BYTE PTR ES:[BX], 33h
11 RET
```



BX = 0010h.

Adım 5 / 9

DEĞİŞKENLER

DS = **2000h** | ES = **3000h** | SS = **0700h** | BX = **0010h** | BP = **0000h**

Bağıl konum aynı olsa da kesim farklıysa erişilen bayt farklıdır: BP içeren adres SS, BX/SI/DI içeren adres DS kullanır.

Kod İzleme: [BX], [BP] ve ES:[BX]

```
1  ORG 100h
2  MOV AX, 2000h
3  MOV DS, AX
4  MOV AX, 3000h
5  MOV ES, AX
6  MOV BX, 0010h
▶  MOV BP, BX        ; aynı ofset
8  MOV BYTE PTR [BX], 11h
9  MOV BYTE PTR [BP], 22h
10 MOV BYTE PTR ES:[BX], 33h
11 RET
```



BP = BX = 0010h. İki yazmaç aynı bağıl konumu tutuyor.

Adım 6 / 9

DEĞİŞKENLER

DS = 2000h | ES = 3000h | SS = 0700h | BX = 0010h | BP = 0010h

Bağıl konum aynı olsa da kesim farklıysa erişilen bayt farklıdır: BP içeren adres SS, BX/SI/DI içeren adres DS kullanır.

Kod İzleme: [BX], [BP] ve ES:[BX]

```
1  ORG 100h
2  MOV AX, 2000h
3  MOV DS, AX
4  MOV AX, 3000h
5  MOV ES, AX
6  MOV BX, 0010h
7  MOV BP, BX       ; aynı ofset
▶  MOV BYTE PTR [BX], 11h
9  MOV BYTE PTR [BP], 22h
10 MOV BYTE PTR ES:[BX], 33h
11 RET
```



`[BX]` → varsayılan kesim **DS**: 20000h + 10h = **20010h** ← 11h.

Adım 7 / 9

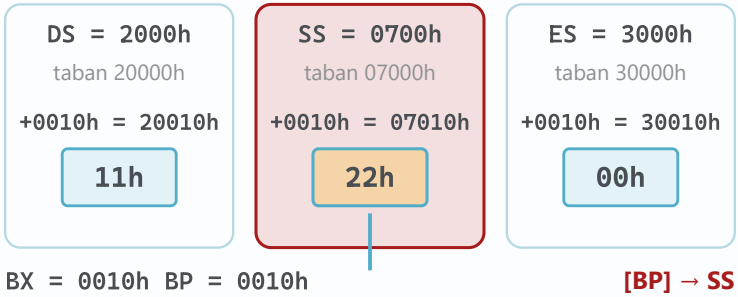
DEĞİŞKENLER

DS = **2000h** | ES = **3000h** | SS = **0700h** | BX = **0010h** | BP = **0010h**

Bağıl konum aynı olsa da kesim farklıysa erişilen bayt farklıdır: BP içeren adres SS, BX/SI/DI içeren adres DS kullanır.

Kod İzleme: [BX], [BP] ve ES:[BX]

```
1  ORG 100h
2  MOV AX, 2000h
3  MOV DS, AX
4  MOV AX, 3000h
5  MOV ES, AX
6  MOV BX, 0010h
7  MOV BP, BX        ; aynı ofset
8  MOV BYTE PTR [BX], 11h
▶  MOV BYTE PTR [BP], 22h
10 MOV BYTE PTR ES:[BX], 33h
11 RET
```



`[BP]` → varsayılan kesim **SS**: 07000h + 10h = **07010h** ← 22h. Aynı ofset, başka bayt!

Adım 8 / 9

DEĞİŞKENLER

DS = **2000h** | ES = **3000h** | SS = **0700h** | BX = **0010h** | BP = **0010h**

Bağıl konum aynı olsa da kesim farklıysa erişilen bayt farklıdır: BP içeren adres SS, BX/SI/DI içeren adres DS kullanır.

Kod İzleme: [BX], [BP] ve ES:[BX]

```
1  ORG 100h
2  MOV AX, 2000h
3  MOV DS, AX
4  MOV AX, 3000h
5  MOV ES, AX
6  MOV BX, 0010h
7  MOV BP, BX        ; aynı ofset
8  MOV BYTE PTR [BX], 11h
9  MOV BYTE PTR [BP], 22h
▶ MOV BYTE PTR ES:[BX], 33h
11 RET
```



`ES:` öneki varsayılanı değiştirir: 30000h + 10h = **30010h** ← 33h.

Adım 9 / 9

DEĞİŞKENLER

DS = **2000h** · ES = **3000h** · SS = **0700h** · BX = **0010h** · BP = **0010h**

Bağıl konum aynı olsa da kesim farklıysa erişilen bayt farklıdır: BP içeren adres SS, BX/SI/DI içeren adres DS kullanır.

Özel Amaçlı Yazmaçlar

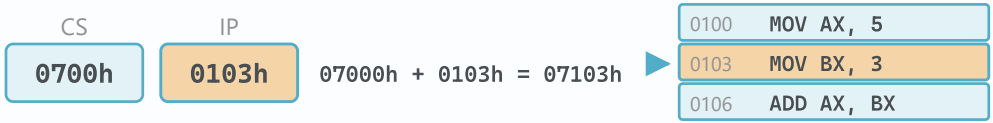
Mikroişlemcinin belirli görevleri için ayrılmıştır; genel amaçlı yazmaçlar gibi **MOV ile doğrudan** erişilemez.

IP · Instruction Pointer

- CS ile birlikte çalışır: `CS:IP`, **getirilecek bir sonraki komutu** gösterir.
- Diğer işlemcilerde *Program Counter* (PC) olarak da bilinir.
- Her komut getirildiğinde komut uzunluğu kadar artar.
- Yalnızca JMP, CALL, RET, INT, IRET ve koşullu atlamalar değiştirir.

FLAGS · Bayrak yazmacı

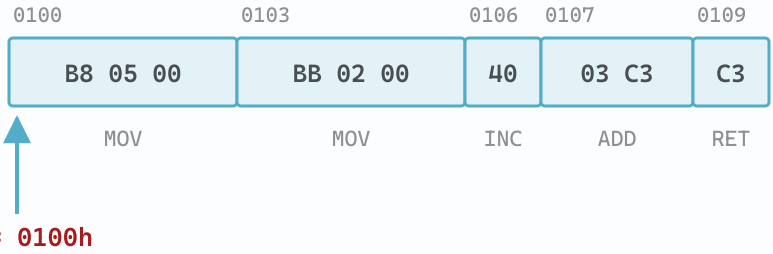
- İşlemlerden sonra CPU tarafından **otomatik** güncellenir.
- Koşullu atlamalarla programın akışını (kontrolü) diğer bölümlere aktarmak için kullanılır.
- Sonucun durumunu gösteren bitlere **durum bayrakları**, CPU'yu yönlendiren bitlere **kontrol bayrakları** denir.



Not: 8086'da IP, o anda yürütülen komutu değil, **sıradaki** komutu gösterir (komut kuyruğu nedeniyle daha da ileride okuma yapılabilir).

Kod İzleme: IP Nasıl İlerler?

```
▶ ORG 100h
2 MOV AX, 5      ; 3 bayt
3 MOV BX, 2      ; 3 bayt
4 INC AX         ; 1 bayt
5 ADD AX, BX     ; 2 bayt
6 RET            ; 1 bayt
```



Program CS:0100h'ye yüklenir; **IP = 0100h** ilk komutu gösterir.

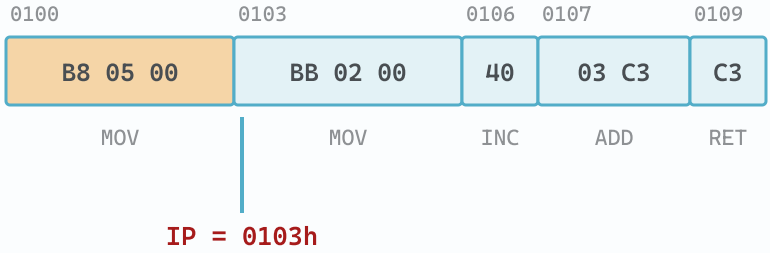
Adım 1 / 6

DEĞİŞKENLER

IP = **0100h** AX = **0000h** BX = **0000h** SP = **FFFEh**

Kod İzleme: IP Nasıl İlerler?

```
1  ORG 100h
▶  MOV AX, 5      ; 3 bayt
3  MOV BX, 2      ; 3 bayt
4  INC AX         ; 1 bayt
5  ADD AX, BX     ; 2 bayt
6  RET            ; 1 bayt
```



MOV AX, 5 (B8 05 00) getirilir: **IP = 0103h**, yürütülünce AX = 0005h.

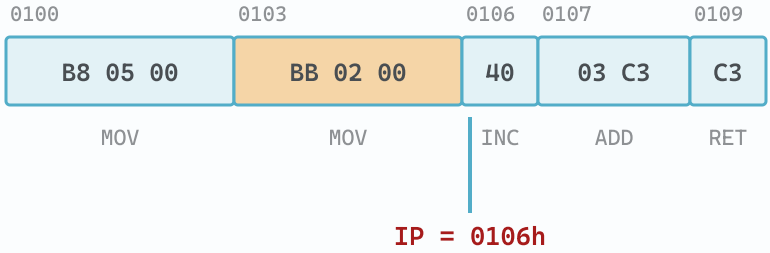
Adım 2 / 6

DEĞİŞKENLER

IP = 0103h | AX = 0005h | BX = 0000h | SP = FFFEh

Kod İzleme: IP Nasıl İlerler?

```
1  ORG 100h
2  MOV AX, 5       ; 3 bayt
▶  MOV BX, 2       ; 3 bayt
4  INC AX          ; 1 bayt
5  ADD AX, BX      ; 2 bayt
6  RET             ; 1 bayt
```



3 bayt daha: **IP = 0106h**, BX = 0002h.

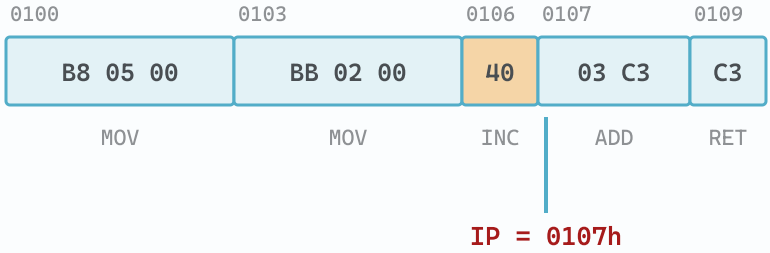
Adım 3 / 6

DEĞİŞKENLER

IP = 0106h | AX = 0005h | BX = 0002h | SP = FFFEh

Kod İzleme: IP Nasıl İlerler?

```
1 ORG 100h
2 MOV AX, 5      ; 3 bayt
3 MOV BX, 2      ; 3 bayt
▶ INC AX         ; 1 bayt
5 ADD AX, BX     ; 2 bayt
6 RET            ; 1 bayt
```



INC AX tek bayttır (40h): **IP = 0107h**, AX = 0006h.

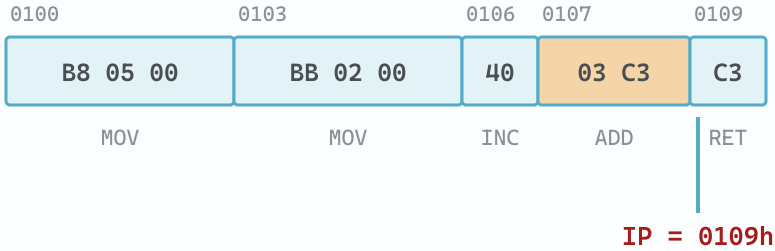
Adım 4 / 6

DEĞİŞKENLER

IP = 0107h | AX = 0006h | BX = 0002h | SP = FFFEh

Kod İzleme: IP Nasıl İlerler?

```
1 ORG 100h
2 MOV AX, 5      ; 3 bayt
3 MOV BX, 2      ; 3 bayt
4 INC AX         ; 1 bayt
▶ ADD AX, BX     ; 2 bayt
6 RET            ; 1 bayt
```



ADD AX, BX 2 bayt: **IP = 0109h**, AX = 0008h.

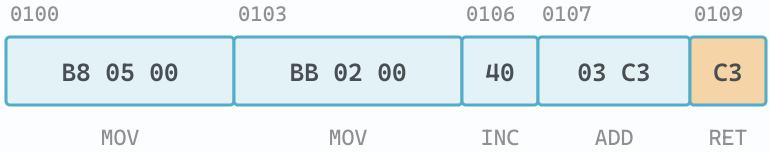
Adım 5 / 6

DEĞİŞKENLER

IP = 0109h | AX = 0008h | BX = 0002h | SP = FFFEh

Kod İzleme: IP Nasıl İlerler?

```
1 ORG 100h
2 MOV AX, 5      ; 3 bayt
3 MOV BX, 2      ; 3 bayt
4 INC AX         ; 1 bayt
5 ADD AX, BX     ; 2 bayt
▶ RET            ; 1 bayt
```



IP = 0000h (PSP'ye dönüş)

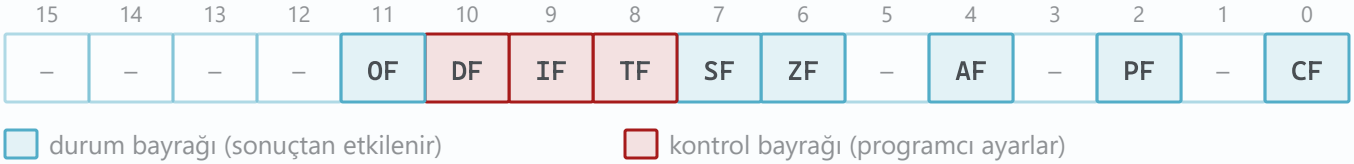
RET yığından 0000h'yi çeker: **IP = 0000h**, SP = 0000h. PSP:0000'daki INT 20h programı bitirir.

Adım 6 / 6

DEĞİŞKENLER

IP = 0000h | AX = 0008h | BX = 0002h | SP = 0000h

Bayrak Yazmacı



Bayrak	Bit	1 olur, eğer…
`CF` elde (carry)	0	işaretsiz sonuç sığmadıysa: toplamada elde, çıkarmada ödünç
`PF` eşlik (parity)	2	sonucun **düşük baytında** çift sayıda 1 varsa
`AF` yardımcı elde	4	bit 3'ten bit 4'e elde/ödünç olduysa (BCD işlemleri)
`ZF` sıfır (zero)	6	sonuç sıfırsa
`SF` işaret (sign)	7	sonucun en anlamlı biti 1 ise (negatif)
`OF` taşma (overflow)	11	işaretli sonuç sığmadıysa
`TF`, `IF`, `DF`	8, 9, 10	adım adım çalışma, kesmelere izin, dizgi yönü (kontrol)

Bayraklar Nasıl Oluşur?

`MOV AL, 7Fh` · `MOV BL, 01h` · `ADD AL, BL`

AL	`0111 1111`	`7Fh`	+127
+ BL	`0000 0001`	`01h`	+1
= AL	`1000 0000`	`80h`	işaretsiz 128, işaretli −128

CF = 0 · bit 7'den dışarı elde çıkmadı (işaretsiz 127 + 1 = 128 sığar).

Bayraklar Nasıl Oluşur?

`MOV AL, 7Fh` · `MOV BL, 01h` · `ADD AL, BL`

AL	`0111 1111`	`7Fh`	+127
+ BL	`0000 0001`	`01h`	+1
= AL	`1000 0000`	`80h`	işaretsiz 128, işaretli −128

CF = 0 · bit 7'den dışarı elde çıkmadı (işaretsiz 127 + 1 = 128 sığar).

ZF = 0 · sonuç 80h sıfır değil.

Bayraklar Nasıl Oluşur?

`MOV AL, 7Fh` · `MOV BL, 01h` · `ADD AL, BL`

AL	`0111 1111`	`7Fh`	+127
+ BL	`0000 0001`	`01h`	+1
= AL	`1000 0000`	`80h`	işaretsiz 128, işaretli −128

CF = 0 · bit 7'den dışarı elde çıkmadı (işaretsiz 127 + 1 = 128 sığar).

ZF = 0 · sonuç 80h sıfır değil.

SF = 1 · sonucun bit 7'si 1 → işaretli yorumda negatif.

Bayraklar Nasıl Oluşur?

`MOV AL, 7Fh` · `MOV BL, 01h` · `ADD AL, BL`

AL	`0111 1111`	`7Fh`	+127
+ BL	`0000 0001`	`01h`	+1
= AL	`1000 0000`	`80h`	işaretsiz 128, işaretli −128

CF = 0 · bit 7'den dışarı elde çıkmadı (işaretsiz 127 + 1 = 128 sığar).

ZF = 0 · sonuç 80h sıfır değil.

SF = 1 · sonucun bit 7'si 1 → işaretli yorumda negatif.

OF = 1 · iki pozitif sayının toplamı negatif çıktı → işaretli taşma (+128, 8 bite sığmaz).

Bayraklar Nasıl Oluşur?

`MOV AL, 7Fh` · `MOV BL, 01h` · `ADD AL, BL`

AL	`0111 1111`	`7Fh`	+127
+ BL	`0000 0001`	`01h`	+1
= AL	`1000 0000`	`80h`	işaretsiz 128, işaretli −128

CF = 0 · bit 7'den dışarı elde çıkmadı (işaretsiz 127 + 1 = 128 sığar).

ZF = 0 · sonuç 80h sıfır değil.

SF = 1 · sonucun bit 7'si 1 → işaretli yorumda negatif.

OF = 1 · iki pozitif sayının toplamı negatif çıktı → işaretli taşma (+128, 8 bite sığmaz).

PF = 0 · 1000 0000'da bir adet 1 var (tek sayı).

Bayraklar Nasıl Oluşur?

`MOV AL, 7Fh` · `MOV BL, 01h` · `ADD AL, BL`

AL	`0111 1111`	`7Fh`	+127
+ BL	`0000 0001`	`01h`	+1
= AL	`1000 0000`	`80h`	işaretsiz 128, işaretli −128

CF = 0 · bit 7'den dışarı elde çıkmadı (işaretsiz 127 + 1 = 128 sığar).

ZF = 0 · sonuç 80h sıfır değil.

SF = 1 · sonucun bit 7'si 1 → işaretli yorumda negatif.

OF = 1 · iki pozitif sayının toplamı negatif çıktı → işaretli taşma (+128, 8 bite sığmaz).

PF = 0 · 1000 0000'da bir adet 1 var (tek sayı).

AF = 1 · düşük nibble: Fh + 1h = 10h → bit 3'ten bit 4'e elde.

Bayrak Hesaplayıcı

AL = 7F h ADD BL = 01 h

7Fh + 01h = 80h (10000000b)

işaretsiz: 127 + 1 = 128 → 128 · işaretli: 127 + 1 = 128 → −128

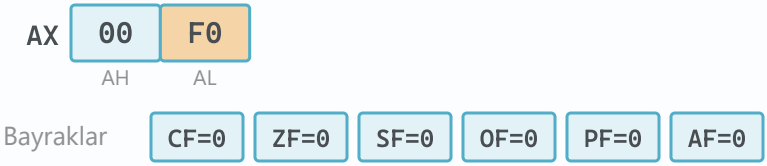
CF	ZF	SF	OF	PF	AF
0	0	1	1	0	1

CF: işaretsiz toplam 255'i aştı mı? · **OF**: işaretli sonuç −128…127 dışında mı? · **PF**: sonuçta 1 adet 1 → tek · **AF**: alt 4 bitte elde var

8 bitlik `ADD AL, BL` ya da `SUB AL, BL` sonrası durum bayrakları. Değerleri onaltılık girin (00–FF).

Kod İzleme: Bayraklar

```
▶ MOV AL, 0F0h
2 ADD AL, 20h     ; F0h + 20h
3 SUB AL, 10h
4 SUB AL, 1
5 RET
```



AL = F0h (işaretsiz 240, işaretli −16). MOV bayrakları değiştirmez.

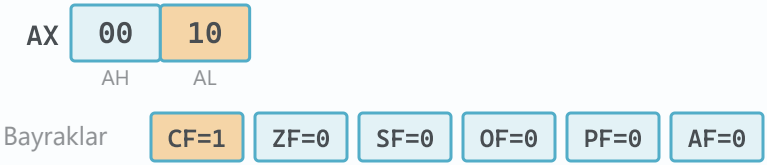
Adım 1 / 5

DEĞİŞKENLER

AL = **F0h** | CF = **0** | ZF = **0** | SF = **0** | OF = **0** | PF = **0** | AF = **0**

Kod İzleme: Bayraklar

```
1 MOV AL, 0F0h
▶ ADD AL, 20h     ; F0h + 20h
3 SUB AL, 10h
4 SUB AL, 1
5 RET
```



F0h + 20h = 110h → AL = **10h**, dışarı taşan 1 → **CF = 1**. İşaretli: −16 + 32 = 16 doğru → OF = 0.

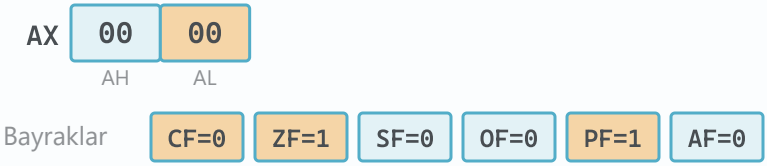
Adım 2 / 5

DEĞİŞKENLER

AL = **10h** | CF = **1** | ZF = **0** | SF = **0** | OF = **0** | PF = **0** | AF = **0**

Kod İzleme: Bayraklar

```
1 MOV AL, 0F0h
2 ADD AL, 20h     ; F0h + 20h
▶ SUB AL, 10h
4 SUB AL, 1
5 RET
```



10h − 10h = 00h → **ZF = 1**, PF = 1 (sıfır adet 1 çifttir), ödünç yok → CF = 0.

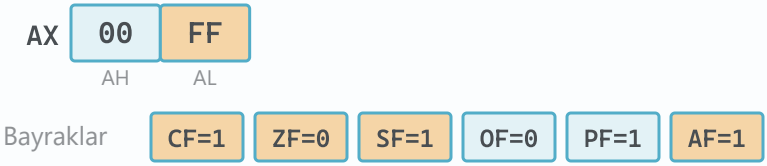
Adım 3 / 5

DEĞİŞKENLER

AL = **00h** · CF = **0** · ZF = **1** · SF = **0** · OF = **0** · PF = **1** · AF = **0**

Kod İzleme: Bayraklar

```
1 MOV AL, 0F0h
2 ADD AL, 20h     ; F0h + 20h
3 SUB AL, 10h
▶ SUB AL, 1
5 RET
```



00h − 1 = **FFh**: ödünç alındı → **CF = 1**, SF = 1 (işaretli −1), PF = 1 (sekiz adet 1), AF = 1.

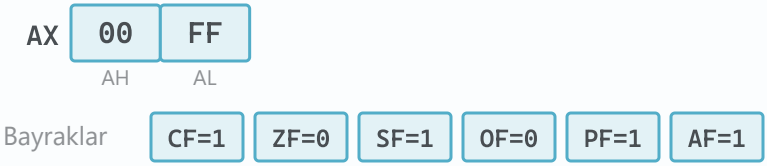
Adım 4 / 5

DEĞİŞKENLER

AL = **FFh** | CF = **1** | ZF = **0** | SF = **1** | OF = **0** | PF = **1** | AF = **1**

Kod İzleme: Bayraklar

```
1 MOV AL, 0F0h
2 ADD AL, 20h     ; F0h + 20h
3 SUB AL, 10h
4 SUB AL, 1
▶ RET
```



RET ile DOS'a dönülür. Aynı bit deseni (FFh) işaretsiz 255, işaretli −1 olarak yorumlanabilir.

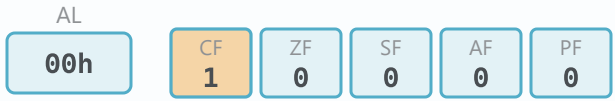
Adım 5 / 5

DEĞİŞKENLER

AL = **FFh** | CF = **1** | ZF = **0** | SF = **1** | OF = **0** | PF = **1** | AF = **1**

Kod İzleme: Hangi Komut Bayrağa Dokunur?

```
1  ORG 100h
▶ STC                ; CF = 1
3  MOV AL, 0FFh      ; bayraklar aynı
4  INC AL            ; CF'ye dokunmaz
5  MOV CX, 0         ; ZF aynı kalır
6  DEC AL
7  ADD AL, 1         ; CF yeni
8  CLC
9  NOT AL            ; bayraklar aynı
10 RET
```



yalnız CF

`STC` : CF = 1. Bayrağı doğrudan değiştiren komutlardan biri.

Adım 1 / 8

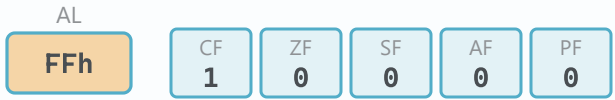
DEĞİŞKENLER

AL = 00h | CF = 1 | ZF = 0 | SF = 0

MOV, NOT, XCHG, LEA, PUSH ve POP bayrakları değiştirmez; INC ve DEC CF dışındakileri değiştirir. Koşullu atlamadan hemen önce bayrağı hangi komutun ayarladığına bakılır.

Kod İzleme: Hangi Komut Bayrağa Dokunur?

```
1  ORG 100h
2  STC              ; CF = 1
▶ MOV AL, 0FFh      ; bayraklar aynı
4  INC AL           ; CF'ye dokunmaz
5  MOV CX, 0        ; ZF aynı kalır
6  DEC AL
7  ADD AL, 1        ; CF yeni
8  CLC
9  NOT AL           ; bayraklar aynı
10 RET
```



bayrak yok

`MOV AL, 0FFh`: AL değişti, bayrakların hiçbiri değişmedi. MOV bayrak etkilemez.

Adım 2 / 8

DEĞİŞKENLER

AL = FFh | CF = 1 | ZF = 0 | SF = 0

MOV, NOT, XCHG, LEA, PUSH ve POP bayrakları değiştirmez; INC ve DEC CF dışındakileri değiştirir. Koşullu atlamadan hemen önce bayrağı hangi komutun ayarladığına bakılır.

Kod İzleme: Hangi Komut Bayrağa Dokunur?

```
1  ORG 100h
2  STC               ; CF = 1
3  MOV AL, 0FFh      ; bayraklar aynı
▶  INC AL            ; CF'ye dokunmaz
5  MOV CX, 0         ; ZF aynı kalır
6  DEC AL
7  ADD AL, 1         ; CF yeni
8  CLC
9  NOT AL            ; bayraklar aynı
10 RET
```



CF korunur

`INC AL` : FFh + 1 = 00h → ZF = 1, AF = 1, PF = 1. CF hâlâ 1: INC ve DEC elde bayrağına dokunmaz.

Adım 3 / 8

DEĞİŞKENLER

AL = 00h | CF = 1 | ZF = 1 | SF = 0

MOV, NOT, XCHG, LEA, PUSH ve POP bayrakları değiştirmez; INC ve DEC CF dışındakileri değiştirir. Koşullu atlamadan hemen önce bayrağı hangi komutun ayarladığına bakılır.

Kod İzleme: Hangi Komut Bayrağa Dokunur?

```
1  ORG 100h
2  STC              ; CF = 1
3  MOV AL, 0FFh     ; bayraklar aynı
4  INC AL           ; CF'ye dokunmaz
▶  MOV CX, 0        ; ZF aynı kalır
6  DEC AL
7  ADD AL, 1        ; CF yeni
8  CLC
9  NOT AL           ; bayraklar aynı
10 RET
```

AL

00h

CF	ZF	SF	AF	PF
1	1	0	1	1

bayrak yok

`MOV CX, 0` : sonuç sıfır olsa da ZF yeniden hesaplanmaz; önceki komutun ZF'si kalır.

Adım 4 / 8

DEĞİŞKENLER

AL = 00h | CF = 1 | ZF = 1 | SF = 0

MOV, NOT, XCHG, LEA, PUSH ve POP bayrakları değiştirmez; INC ve DEC CF dışındakileri değiştirir. Koşullu atlamadan hemen önce bayrağı hangi komutun ayarladığına bakılır.

Kod İzleme: Hangi Komut Bayrağa Dokunur?

```
1  ORG 100h
2  STC               ; CF = 1
3  MOV AL, 0FFh      ; bayraklar aynı
4  INC AL            ; CF'ye dokunmaz
5  MOV CX, 0         ; ZF aynı kalır
▶  DEC AL
7  ADD AL, 1         ; CF yeni
8  CLC
9  NOT AL            ; bayraklar aynı
10 RET
```



CF korunur

`DEC AL` : 00h − 1 = FFh → SF = 1, ZF = 0. CF yine 1: DEC ödünç almayı CF'ye yazmaz.

Adım 5 / 8

DEĞİŞKENLER

AL = FFh | CF = 1 | ZF = 0 | SF = 1

MOV, NOT, XCHG, LEA, PUSH ve POP bayrakları değiştirmez; INC ve DEC CF dışındakileri değiştirir. Koşullu atlamadan hemen önce bayrağı hangi komutun ayarladığına bakılır.

Kod İzleme: Hangi Komut Bayrağa Dokunur?

```
1  ORG 100h
2  STC               ; CF = 1
3  MOV AL, 0FFh      ; bayraklar aynı
4  INC AL            ; CF'ye dokunmaz
5  MOV CX, 0         ; ZF aynı kalır
6  DEC AL
▶  ADD AL, 1         ; CF yeni
8  CLC
9  NOT AL            ; bayraklar aynı
10 RET
```



hepsi yeni

`ADD AL, 1` : FFh + 1 = 100h → AL = 00h. ADD bütün durum bayraklarını yeniden yazar: CF = 1 bu kez ADD'in eldesi.

Adım 6 / 8

DEĞİŞKENLER

AL = 00h | CF = 1 | ZF = 1 | SF = 0

MOV, NOT, XCHG, LEA, PUSH ve POP bayrakları değiştirmez; INC ve DEC CF dışındakileri değiştirir. Koşullu atlamadan hemen önce bayrağı hangi komutun ayarladığına bakılır.

Kod İzleme: Hangi Komut Bayrağa Dokunur?

```
1  ORG 100h
2  STC               ; CF = 1
3  MOV AL, 0FFh      ; bayraklar aynı
4  INC AL            ; CF'ye dokunmaz
5  MOV CX, 0         ; ZF aynı kalır
6  DEC AL
7  ADD AL, 1         ; CF yeni
▶  CLC
9  NOT AL            ; bayraklar aynı
10 RET
```

AL

00h

CF	ZF	SF	AF	PF
0	1	0	1	1

yalnız CF

`CLC` : CF = 0.

Adım 7 / 8

DEĞİŞKENLER

AL = 00h | CF = 0 | ZF = 1 | SF = 0

MOV, NOT, XCHG, LEA, PUSH ve POP bayrakları değiştirmez; INC ve DEC CF dışındakileri değiştirir. Koşullu atlamadan hemen önce bayrağı hangi komutun ayarladığına bakılır.

Kod İzleme: Hangi Komut Bayrağa Dokunur?

```
1  ORG 100h
2  STC               ; CF = 1
3  MOV AL, 0FFh      ; bayraklar aynı
4  INC AL            ; CF'ye dokunmaz
5  MOV CX, 0         ; ZF aynı kalır
6  DEC AL
7  ADD AL, 1         ; CF yeni
8  CLC
▶  NOT AL            ; bayraklar aynı
10 RET
```



bayrak yok

`NOT AL` : AL = FFh, ama ZF hâlâ 1. NOT hiçbir bayrağı değiştirmez; sonucu sınamak için ardından `TEST AL, AL` ya da `CMP` gerekir.

Adım 8 / 8

DEĞİŞKENLER

AL = FFh | CF = 0 | ZF = 1 | SF = 0

MOV, NOT, XCHG, LEA, PUSH ve POP bayrakları değiştirmez; INC ve DEC CF dışındakileri değiştirir. Koşullu atlamadan hemen önce bayrağı hangi komutun ayarladığına bakılır.

İLK PROGRAM

.COM Program Şablonu

```
ORG 100h            ; .COM: kod 0100h'den başlar

MOV AX, 5           ; AX = 5
MOV BX, 3           ; BX = 3
ADD AX, BX          ; AX = AX + BX = 8
MOV CL, AL          ; CL = 08h

RET                 ; DOS'a dön

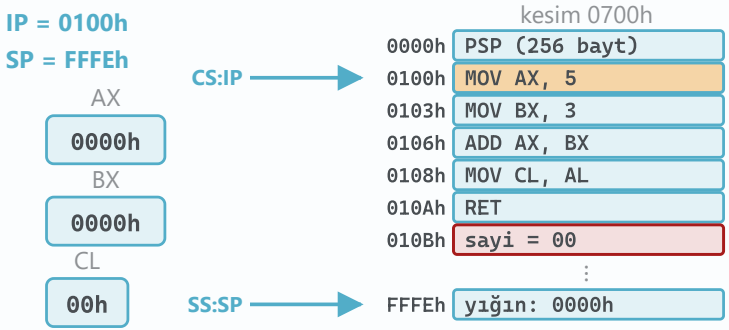
sayi DB 0           ; değişkenler RET'ten sonra
```

- `ORG 100h` : .COM programı kesimin 0100h adresine yüklenir; ilk 256 bayt (PSP) DOS'a aittir.
- Komutlar yukarıdan aşağı, **sırayla** yürütülür.
- `RET` : işletim sistemine geri döner.
- Değişkenler kodun **sonuna** (RET'ten sonra) konur ki CPU onları komut sanıp yürütmesin.
- .COM'da başlangıçta `CS = DS = ES = SS` , `IP = 0100h` , `SP = FFFEh` .

emu8086'da: **new** → **COM template**, kodu yaz, **emulate**; ardından **single step** ile her komutun yazmaçları nasıl değiştirdiğini izle.

Kod İzleme: .COM Programı Bellekte

```
▶ ORG 100h          ; kod 0100h'den
2 MOV AX, 5
3 MOV BX, 3
4 ADD AX, BX        ; AX = 8
5 MOV CL, AL        ; CL = 08h
6 RET               ; DOS'a dön
7 sayi DB 0         ; veri
```



Program 0700:0100'e yüklendi. CS = DS = ES = SS = 0700h, **IP = 0100h**, **SP = FFFEh**; yığın tepesinde 0000h var.

Adım 1 / 6

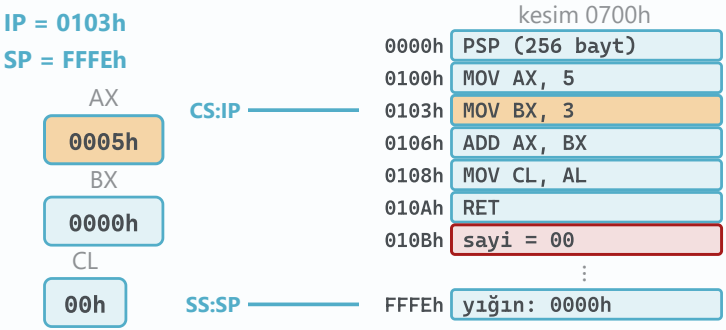
DEĞİŞKENLER

`AX = 0000h` `BX = 0000h` `CL = 00h` `IP = 0100h` `SP = FFFEh`

Komutlar 3, 3, 2, 2 ve 1 bayttır; IP her komutta uzunluğu kadar ilerler. Veriler RET'ten sonra durduğu için komut olarak yürütülmez.

Kod İzleme: .COM Programı Bellekte

```
1  ORG 100h        ; kod 0100h'den
▶  MOV AX, 5
3  MOV BX, 3
4  ADD AX, BX      ; AX = 8
5  MOV CL, AL      ; CL = 08h
6  RET             ; DOS'a dön
7  sayi DB 0       ; veri
```



`MOV AX, 5` (3 bayt): AX = 0005h, IP = **0103h**.

Adım 2 / 6

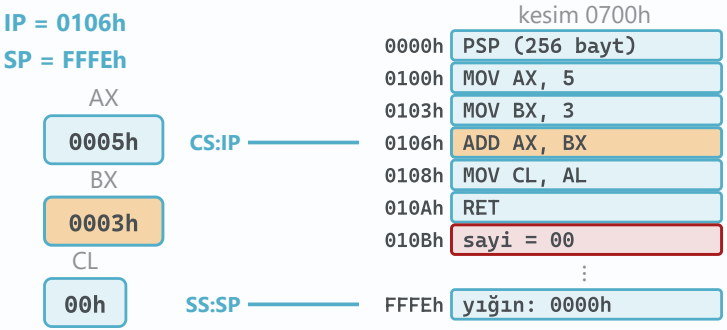
DEĞİŞKENLER

AX = 0005h | BX = 0000h | CL = 00h | IP = 0103h | SP = FFFEh

Komutlar 3, 3, 2, 2 ve 1 bayttır; IP her komutta uzunluğu kadar ilerler. Veriler RET'ten sonra durduğu için komut olarak yürütülmez.

Kod İzleme: .COM Programı Bellekte

```
1 ORG 100h        ; kod 0100h'den
2 MOV AX, 5
▶ MOV BX, 3
4 ADD AX, BX      ; AX = 8
5 MOV CL, AL      ; CL = 08h
6 RET             ; DOS'a dön
7 sayi DB 0       ; veri
```



`MOV BX, 3` (3 bayt): BX = 0003h, IP = **0106h**.

Adım 3 / 6

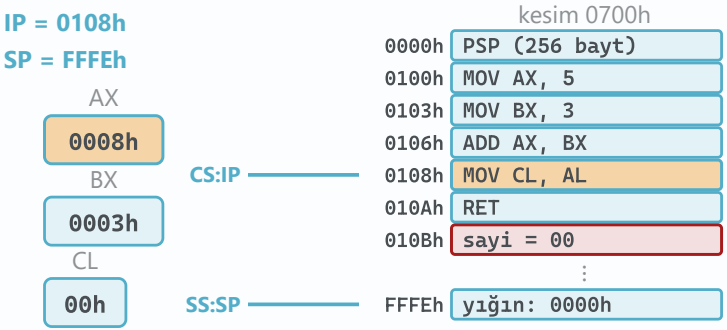
DEĞİŞKENLER

AX = **0005h** · BX = **0003h** · CL = **00h** · IP = **0106h** · SP = **FFFEh**

Komutlar 3, 3, 2, 2 ve 1 bayttır; IP her komutta uzunluğu kadar ilerler. Veriler RET'ten sonra durduğu için komut olarak yürütülmez.

Kod İzleme: .COM Programı Bellekte

```
1  ORG 100h        ; kod 0100h'den
2  MOV AX, 5
3  MOV BX, 3
▶  ADD AX, BX      ; AX = 8
5  MOV CL, AL      ; CL = 08h
6  RET             ; DOS'a dön
7  sayi DB 0       ; veri
```



`ADD AX, BX` (2 bayt): AX = **0008h**, IP = 0108h.

Adım 4 / 6

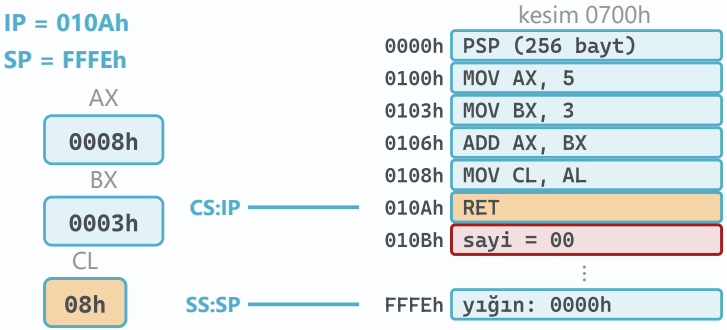
DEĞİŞKENLER

AX = 0008h | BX = 0003h | CL = 00h | IP = 0108h | SP = FFFEh

Komutlar 3, 3, 2, 2 ve 1 bayttır; IP her komutta uzunluğu kadar ilerler. Veriler RET'ten sonra durduğu için komut olarak yürütülmez.

Kod İzleme: .COM Programı Bellekte

```
1 ORG 100h        ; kod 0100h'den
2 MOV AX, 5
3 MOV BX, 3
4 ADD AX, BX      ; AX = 8
▶ MOV CL, AL      ; CL = 08h
6 RET             ; DOS'a dön
7 sayi DB 0       ; veri
```



`MOV CL, AL` (2 bayt): CL = **08h**, IP = 010Ah.

Adım 5 / 6

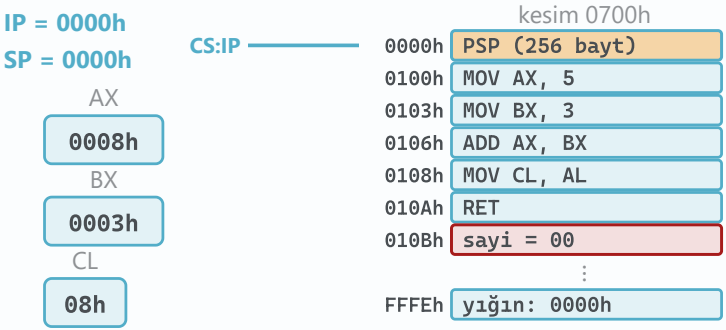
DEĞİŞKENLER

AX = **0008h** | BX = **0003h** | CL = **08h** | IP = **010Ah** | SP = **FFFEh**

Komutlar 3, 3, 2, 2 ve 1 bayttır; IP her komutta uzunluğu kadar ilerler. Veriler RET'ten sonra durduğu için komut olarak yürütülmez.

Kod İzleme: .COM Programı Bellekte

```
1 ORG 100h        ; kod 0100h'den
2 MOV AX, 5
3 MOV BX, 3
4 ADD AX, BX      ; AX = 8
5 MOV CL, AL      ; CL = 08h
▶ RET             ; DOS'a dön
7 sayi DB 0       ; veri
```



`RET` yığından 0000h'yi alır: IP = 0000h, SP = 0000h. PSP:0000'daki `INT 20h` programı bitirir; 010Bh'deki `sayi` hiç yürütülmedi.

Adım 6 / 6

DEĞİŞKENLER

`AX = 0008h` `BX = 0003h` `CL = 08h` `IP = 0000h` `SP = 0000h`

Komutlar 3, 3, 2, 2 ve 1 bayttır; IP her komutta uzunluğu kadar ilerler. Veriler RET'ten sonra durduğu için komut olarak yürütülmez.

İLK PROGRAM

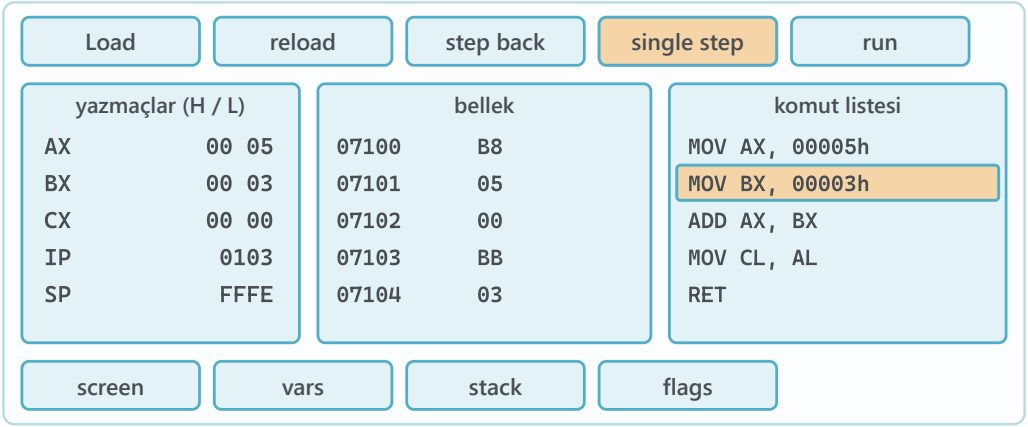
Program Bellekte: Bayt Bayt

Bağıl konum	Baytlar	Kaynak satırı	Not
0100h	`B8 05 00`	`MOV AX, 5`	B8 + yazmaç no (AX = 0); sabit little-endian
0103h	`BB 03 00`	`MOV BX, 3`	B8 + 3 (BX)
0106h	`03 C3`	`ADD AX, BX`	C3 = 11 000 011: yazmaç–yazmaç, AX, BX
0108h	`8A C8`	`MOV CL, AL`	C8 = 11 001 000: CL, AL
010Ah	`C3`	`RET`	tek baytlık komut
010Bh	`00`	`sayi DB 0`	veri; komut değil

- .COM dosyası tam olarak bu 12 bayttır: başlık yoktur, DOS baytları 0100h'den itibaren belleğe kopyalar.
- `ORG 100h` satırının karşılığında bayt yoktur; yalnızca ilk baytın konumunu belirler.
- Aynı komutun birden çok kodlaması olabilir: `ADD AX, BX` için `01 D8` de geçerlidir. MASM 03 C3'ü seçer; ikisi de aynı işi yapar.
- Bellekte komut ile veri arasında hiçbir işaret yoktur: 010Bh'deki 00, IP oraya gelseydi komut olarak çözülürdü.

İLK PROGRAM

emu8086 Öykünücü Penceresi



- single step bir komut yürütür; değişen yazmaçlar ve bellek hemen görünür.
- step back son adımı geri alır; bir hatanın nedenine geri dönmeyi kolaylaştırır.
- Komut listesindeki vurgulu satır **sıradaki** komuttur (CS:IP), az önce yürütülen değil.
- Komut listesi sabitleri onaltılık ve 5 basamakla gösterir (00005h).
- Ek pencereler: ekran çıktısı, değişkenler, yığın ve bayraklar.

Bellek sütunu fiziksel adresleri gösterir: 0700:0100 = 07100h. Çizim pencerenin yalnızca ana bölümlerini gösterir.

İLK PROGRAM

Yazım Kuralları

Sayı tabanları

`25` veya `25d` onluk · `19h` onaltılık · `00011001b` ikili · `31o` sekizli. Hepsi aynı değer: 25.

Harfle başlayan onaltılık

Başına `0` konur: `0FFh`, `0B800h`. `FFh` yazılırsa assembler onu bir *ad* (etiket/değişken) sanır.

Karakterler

`'A'` → ASCII kodu 41h. `MOV CL, 'A'` ile `MOV CL, 41h` aynı makine kodunu üretir.

İşaretli sayılar

`-1` 8 bitte FFh, 16 bitte FFFFh olarak saklanır (ikiye tümleyen).

Büyük-küçük harf

Assembler duyarsızdır: `mov`, `MOV`, `Mov` aynıdır; `var1` ile `VAR1` aynı addır.

Açıklamalar

`;` sonrası açıklamadır. Her satıra “ne” değil “neden” yazmak iyi alışkanlıktır.

Sık Yapılan Hatalar

Hatalı	Neden?	Doğrusu
`MOV AL, BX`	Boyutlar farklı (8 ↔ 16 bit)	`MOV AL, BL` veya `MOV AX, BX`
`MOV DS, 1234h`	Kesim yazmacına sabit atanamaz	`MOV AX, 1234h` · `MOV DS, AX`
`MOV AL, [AX]`	Adreslemede yalnız BX, SI, DI, BP	`MOV BX, AX` · `MOV AL, [BX]`
`MOV AL, FFh`	FFh bir ad sanılır	`MOV AL, 0FFh`
`MOV IP, 200h`	IP'ye MOV ile yazılamaz	`JMP` ile atla
`MOV AL, 300`	300 > 255, 8 bite sığmaz	`MOV AX, 300`

Bu kurallar MOV komutunun işlenen kısıtlarından gelir.

Daha Fazla Söz Dizimi Hatası

Hatalı	Neden?	Doğrusu
`MOV CS, AX`	CS yalnız JMP, CALL, RET, INT ile değişir	uzak atlama
`MOV ES, DS`	kesimden kesime MOV yok	`MOV AX, DS` · `MOV ES, AX`
`MOV [BX], [SI]`	iki bellek işleneni	`MOV AL, [SI]` · `MOV [BX], AL`
`INC [BX]`	boyut belirsiz	`INC BYTE PTR [BX]`
`MOV AL, [BX+BP]`	iki taban yazmacı birlikte olmaz	`[BX+SI]` ya da `[BP+DI]`
`ADD 5, AX`	hedef sabit olamaz	`ADD AX, 5`
`PUSH 5`	80186'dan önce yok	`MOV AX, 5` · `PUSH AX`
`SHL AX, 2`	8086'da sayı yalnız 1 ya da CL	`MOV CL, 2` · `SHL AX, CL`

PUSH sabit ve SHL AX, 2 sonraki işlemcilerde geçerlidir; 8086 için yazılan programda assembler hata verir.

İLK PROGRAM

Derleme Hatası mı, Mantık Hatası mı?

Derleme (söz dizimi) hatası

Assembler bulur, satır numarasını verir, program hiç oluşmaz.

- `MOV AL, BX` : boyut uyuşmazlığı
- `JMP bsla` : tanımsız etiket
- `MOV AL, 300` : sığmayan sabit

Mantık hatası

Program derlenir ve çalışır, ama sonuç yanlıştır; assembler bunu bilemez.

- RET'i unutmak: veri komut sanılır
- İşaretli sayı için `JA` kullanmak
- ADC yerine ADD: elde kaybolur
- .EXE'de DS'yi yüklememek

Mantık hatası tek adım çalıştırmayla bulunur: her komuttan sonra beklenen yazmaç ve bayrak değeri öykünücüdeki değerle karşılaştırılır. Değerlerin ayrıldığı ilk komut, hatanın bulunduğu yerdir.

ÖZET

Özet

Assembly
Makine koduyla birebir eşleşen düşük seviyeli dil; küçük ve hızlı kod, donanıma doğrudan erişim, ama taşınabilir değil.

Bilgisayar modeli
CPU + RAM + G/Ç, sistem yolu ile bağlı: adres (20 bit), veri (16 bit), denetim.

Genel amaçlı yazmaçlar
AX, BX, CX, DX (8 bitlik H/L yarılarıyla), SI, DI, BP, SP; hepsi 16 bit.

Kesim yazmaçları
CS, DS, ES, SS; fiziksel adres = kesim × 10h + bağıl konum.

IP
CS:IP sıradaki komutu gösterir; komut uzunluğu kadar ilerler.

Bayraklar
CF, PF, AF, ZF, SF, OF sonuca göre; TF, IF, DF kontrol için.

ÖZET

Alıştırmalar

1. AX = 0ABCDh iken `MOV AH, 0` sonrası AX, AL ve BX değerleri nedir?
2. DS = 2000h, BX = 1234h ise `[BX]` hangi fiziksel adrestir? Aynı adresi veren başka bir kesim:bağıl konum çifti yazın.
3. Fiziksel adres B815Eh için normalize edilmiş kesim:bağıl konum nedir?
4. AL = 80h iken `ADD AL, 80h` sonrası AL, CF, ZF, SF, OF?
5. Program 0100h'de başlar: 3 + 2 + 1 + 4 baytlık dört komut yürütülünce IP ne olur?
6. `MOV CX, AL` ve `MOV ES, 0B800h` neden hatalıdır? Düzeltin.

▶ **Cevaplar**

ÖZET

Kaynaklar

- emu8086 belgeleri, *8086 assembler tutorial for beginners* (bölüm 1: numbering systems, bölüm 2: memory access).
- Intel, *8086 16-Bit HMOS Microprocessor* veri sayfası.
- Barry B. Brey, *The Intel Microprocessors*, 8. baskı, Pearson.
- Muhammad Ali Mazidi, Janice G. Mazidi, *The 80x86 IBM PC and Compatible Computers*, Pearson.
- Andrew S. Tanenbaum, *Modern Operating Systems*, 3. baskı (bilgisayar modeli).

Son

Bölüm 3: Assembly

Sonraki bölüm: Bölüm 4: Bellek Erişimi · Terim sözlüğü